Falcon DDT: system architecture and test plan

6 November 2026 · Falcon FS-AI DDT entry on the IMechE ADS-DV · companion files in the team repository: the compute options and the two lidar pages, and the cost workbook `ddt_essential_costs.xlsx`

In one screen

Few DDT teams score

Of about 36 DDT teams that entered FS-AI from 2022 to 2026, 17 scored in at least one dynamic event and 12 finished a full 10-lap trackdrive. Results.

Teams without a car can finish

At least 4 of those 12 ran the shared IMechE car: IIT Bombay, Trinity, Ain Shams and Bath. All four built or used a simulator early; IIT Bombay and Trinity also had something physical. Teams.

Simulation alone is a gamble

UWE-AI, HWU Dubai (CARLA) and OzU (CarMaker) scored no dynamic points in 2024–26. UWE named perception failures on the real hardware: the gap a sensor rig closes.

Five layers

Vehicle, Perception, Localisation, Planning + control, System. The sensors sit inside the vehicle layer, each next to its driver. Commands reach the car by one path only: controller → CAN bridge → VCU.

Two vehicle profiles

`ADS-DV` for simulation, the rig, the HiL and the event. `1:10` for the scaled car. Same code; the profile swaps the vehicle layer's drivers and sets the numbers.

Three stages, then the event

1 full simulation. 2a pushable rig (real sensors). 2b CarMaker HiL + CARLA (real CAN, localisation against exact truth). 3 the 1:10 car driven over Wi-Fi by the simulated stack. Each closes a gap that cost other teams. Table.

The vehicle model exists

The official HiL repository holds the organisers' 2026 CarMaker model of the ADS-DV, its VCU and the CAN database. The tyre data is the weak spot. Model.

One industrial computer, no GPU

An Advantech MIO-5379 (Core Ultra 7 255H) runs the whole stack on its CPU, Arc graphics and NPU, from the car’s 13.7 V directly. A Jetson Orin NX is an option, only if the camera turns out to need the full ZED SDK. Computer.

Hesai ATX, MTi-8 kit, ZED2

One 120° lidar on the car; the VLP-16 stays on the rig. The Xsens MTi-8 kit brings its own ZED-F9P for RTK. The owned ZED2 gives colour, which the car needs: orange cones mark the start, finish and exit lanes. Hardware.

One clock

The F9P’s PPS sets the computer’s clock, the computer passes it to the lidar over gPTP, and the MTi-8 takes the same PPS on its kit, so lidar points, IMU samples and GNSS fixes share GNSS time. Time.

Cost, landed in Sri Lanka

Essential: LKR 2.12 M for the car kit, the HiL bench adapter and the test rig. Good to have: LKR 0.04 M more. The Orin NX option adds LKR 0.45 M. Cones are made in-house. Cost.

Context: FS-AI results and teams without a car

FS-AI dynamic results, 2022–2026 (checked against the official files)

Counts are teams that finished at least one run in each event. Finishing slower than the time limit still earns 25 points, and trackdrive pays 10 points per completed lap, so any score above zero means the car got round. “10 laps” means at least 100 trackdrive points. Sources: official IMechE results for 2022, 2023, 2024, 2025, 2026.

Year	Entries	Scored any dynamic	Accel	Skidpad	Sprint	Trackdrive, any lap	Trackdrive, 10 laps	Teams that finished all 10 laps
2022	7	4	4	2	2	2	**2**	University of Edinburgh, Coventry University
2023	19	5	2	1	2	5	**3**	The University of Edinburgh, Oxford Brookes University, Edinburgh Napier University
2024	20	7	3	1	2	6	**6**	Edinburgh University Formula Student Driverless, Oxford Brookes Racing Autonomous, UGRacing, UPMRACING, Formula Student Team Napier, UVigo Motorsport (ADS)
2025	25	8	1	2	3	8	**6**	The University of Edinburgh, Oxford Brookes University, University of Glasgow, Indian Institute of Technology Bombay, University of Bath, UPM Technical University of Madrid
2026	27	14	5	2	8	13	**6**	Oxford Brookes University (+APC), University of Glasgow, University of the Basque Country (UPV/EHU) (+APC), Ain Shams University, Trinity College Dublin, University of Strathclyde

2021 had no dynamic events (all DDT teams scored zero there). 2022 lists the DDT class only; two separate ADS entries scored no dynamic points. From 2024 the table includes ADS entries; UVigo’s 2024 trackdrive was with its own ADS car.

Corrections to the DV_feasibility_summary.xlsx Dasun gave me.

- **Sheet3, 2022:** the official file lists 7 DDT teams (place 4 is missing), not 8. Two teams finished acceleration and nothing else (Oxford Brookes, Glasgow), not one.
- **Sheet3, 2025:** “only two events” is 1 (Edinburgh: skidpad and trackdrive) and “only one event” is 6, not 2 and 5.
- **Sheet3, 2026:** “only one event” is 10, not 9. ECE Paris finished acceleration only.
- **Sheet3, every year:** the “Track Drive” column counts any trackdrive lap. Teams that finished all 10 laps: 2, 3, 6, 6 and 6.
- **Sheet2:** the places for IIT Bombay, UWE-AI and Trinity are correct. Trinity’s scaled vehicle (“Rigby”) exists, but its own documentation says no physical motion has been validated since its last audit. The Formula Bharat result for IIT Bombay was not checked.
- **Sheet1:** the ZED2 sees 110° horizontally (120° is the diagonal figure). The VLP-16’s 100 m range figure is right.

How teams without a car got there

Team	Result	How they tested without a car	Stack	Source
IIT Bombay	2025: 4th of 25, full trackdrive	EUFS sim and FSDS for every subsystem. Three versions of a home-built scaled robot with real sensors, driven in poor visibility and with water sprayed on the track. Images of about 3,500 cones taken during test runs on the ADS-DV, used for training.	VLP-16, two ZED2i, SBG Ellipse-N. YOLOv5 with lidar, stereo and mono pipelines in parallel. EKF-SLAM with JCBB association, plus MRPT. Delaunay mid-line. Stanley and pure pursuit. AGX Orin.	Rampuria 2024
Trinity College Dublin	2026: 5th of 27, full trackdrive, sprint finished	"Rigby", a small test vehicle that imitates the car side of the CAN protocol (still being debugged). EUFS sim and IPG CarMaker in its guides. EUFS `ros_can` to talk to the car.	VLP-16 and ZED2, YOLOv5 boxes fused with lidar. GraphSLAM from 2025/26, FastSLAM before that. A GPS with its own EKF. Pure pursuit and a PID speed loop. Jetson AGX Orin.	Rigby, state estimation, perception, Brainy
Ain Shams (Egypt)	2026: 4th of 27, full trackdrive. Best simulation score in 2025 and second best in 2026.	Its 2024 paper says the ADS-DV is "unavailable in our home country", so it trained on a self-built simulator and had little time on the real car at the event.	Not published in detail.	ASU Racing 2024 (abstract only; the full text is not accessible)
Bath (TBRe AI)	2025: 5th, full trackdrive. 2026: won Acceleration and finished all four events.	Ran "the organisers' ADS-DV". Wrote its path planning from scratch, plus a simulator to test it away from the track.	Not published.	Relec 2026
Teams that relied on simulation, for contrast				
UWE-AI	2024: 15th; 2025: 14th; no dynamic points	Felt ready in simulation but it "did not carry over". Blames missing feedback control and perception failures on the supplied hardware. Then built a Raspberry Pi box that emulates the ADS-DV over CAN (lent to Bristol) and a small Ackermann testbed.	—	DesignSpark 2025
HWU Dubai	2025: 7th; no dynamic points (the 2026 entry "Heriot-Watt University" was 14th, also with none)	CARLA, with a stock CARLA car rather than an ADS-DV model.	RoboSense Helios 16P, ZED2i, CHCNAV INS. YOLOv8 and DBSCAN fusion, Cartographer, pure pursuit and PID. AGX Orin 64 GB.	Abdo 2025
OzU Racing	2025: 25th; 2026: 17th; no dynamic points	IPG CarMaker with the ADS-DV model. Its slides say real ADS-DV testing costs track rental and logistics, while simulation is free.	YOLO12 with depth lookup. EKF-SLAM with Hungarian association. ft-fsd-path-planning. Adaptive pure pursuit.	OzU CarMaker deck
Bristol	2026: 18th; 3 trackdrive laps; also brought an APC car	EUFS sim2 in Docker.	Stereo camera only, no lidar.	GitHub (its sim2 Docker repository is no longer public)

The pattern matches the advice of the EUFS software lead from 2021–23: build acceleration first, keep the stack small, use `eufs_sim`, and spend your limited ADS-DV time testing software and recording data, not writing interface code (How to FS-AI). Trackdrive is "autocross repeated 10 times", so it rewards reliability.

System architecture

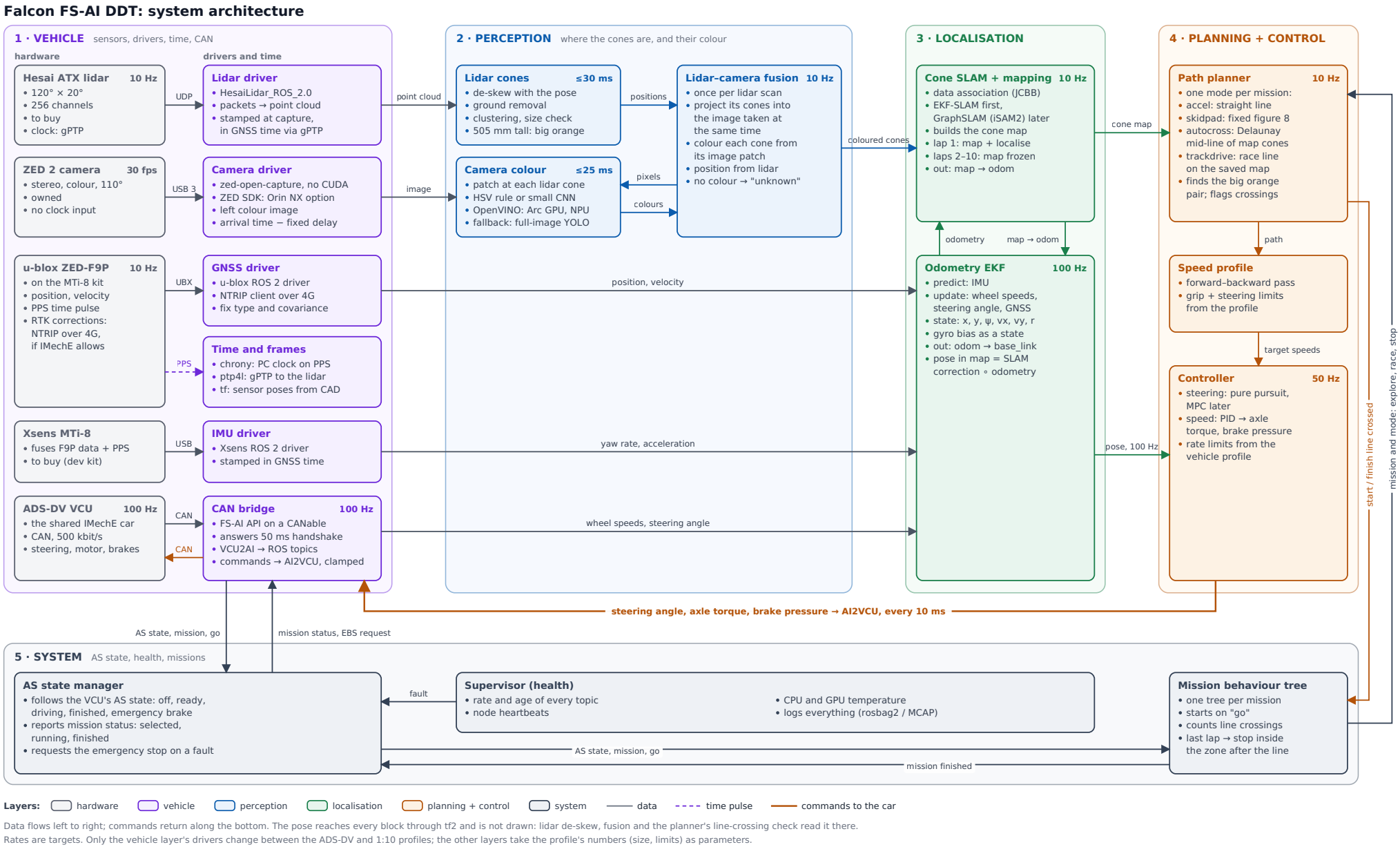


Figure 1. Grey boxes are hardware.

The five layers

Layer	Blocks	In → out	Why it is its own layer
1 · Vehicle	the lidar driver (Hesai's ROS 2 driver, points stamped at capture), the camera driver (`zed-open-capture`; the ZED SDK only with the Orin NX option), GNSS and IMU drivers; time and frames (chrony on the F9P's PPS, gPTP to the lidar, sensor positions from the CAD); CAN bridge (FS-AI API on the CANable, answers the 50 ms handshake)	raw packets, CAN frames → time-stamped ROS 2 messages; commands → AI2VCU	The only layer that touches hardware. Swapping it is how one stack runs in simulation, on the rig, on the HiL, on the 1:10 car and on the ADS-DV.
2 · Perception	lidar cones (crop, de-skew, ground removal, clustering, size check; a 505 mm cone is a big orange); camera colour (a patch of the image at each lidar cone, named by an HSV rule or a small CNN on OpenVINO; full-image YOLO as the fallback); lidar–camera fusion, once per lidar scan	point clouds, image → cones with position, colour (blue, yellow, orange, big orange) and covariance, 10 Hz	It needs real sensors and real cones to test, so it is tested on the rig.
3 · Localisation	odometry EKF (100 Hz); cone SLAM and mapping (EKF-SLAM first, GraphSLAM later)	IMU, GNSS, wheel speeds, steering angle, coloured cones → pose in the map at 100 Hz, cone map	Its metrics need a ground truth: RTK on the rig, the simulator's state in 1 and 2b.
4 · Planning + control	path planner (one mode per mission; plans on map cones, and flags each crossing of the big-orange start and finish lines); speed profile; controller (pure pursuit + PID, MPC later)	cone map, pose → steering, torque, brake; line crossings to the behaviour tree	It needs a vehicle to close the loop: the simulator, the CarMaker model, or the 1:10 car.
5 · System	AS state manager; supervisor; mission behaviour tree	AS state, mission, go, health, line crossings → mission status, emergency-stop request, planner mode	Safety and mission logic stay out of the driving code, so they can be tested with faults on purpose.

How the system layer splits the work.

- **The VCU owns the AS state machine.** Ours only follows it. The CAN database's states are `AS_OFF`, `AS_READY`, `AS_DRIVING`, `EMERGENCY_BRAKE` and `AS_FINISHED`. We report `MISSION_STATUS` (not selected, selected, running, finished) and can set `ESTOP_REQUEST`.
- **The behaviour tree** runs only in `AS_DRIVING`, with one tree per mission. It counts the line crossings the planner reports and, after the last one, sets the planner to stop inside the zone after the line (Colour).
- **The supervisor** can stop the car, through the AS state manager, but it never steers.

How our stack compares with other teams

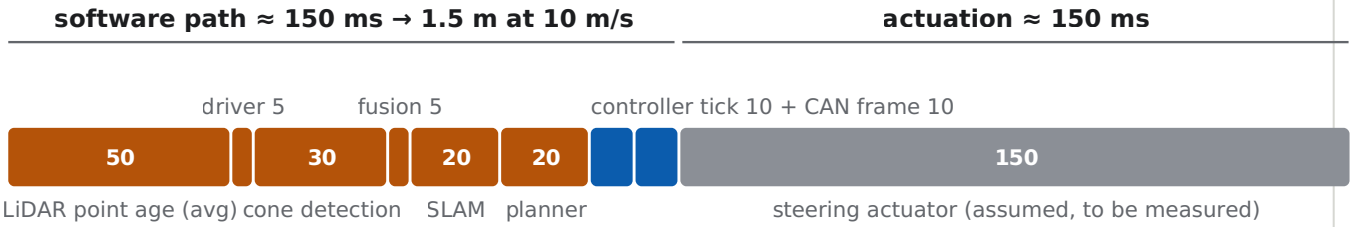
Almost every team writes its own implementation of a few standard algorithms. The libraries that recur are GTSAM, MRPT, Cartographer and `robot_localization`. The FSG column is from the self-reported "Driverless System" field on the team pages of the FSG Driverless Cup: the latest 2024–26 entry for 56 teams. Counts are mentions, so they are lower bounds.

Layer	FS-AI DDT teams (ADS-DV)	FSG Driverless Cup, 56 teams	Falcon
Cone sensor	VLP-16 or another 16-ring lidar, plus a ZED camera	Ouster 17, Hesai 8, RoboSense 6, Livox 2, Velodyne 1. 20 teams list no camera.	Hesai ATX (120° × 20°, 256 channels); the owned ZED2 for colour

Motion	SBG or CHCNAV INS; wheel speeds from the car's CAN	Optical ground-speed sensor 15, Xsens 8, VectorNav 4, SBG 4	Xsens MTi-8 kit with its ZED-F9P, CAN wheel speeds (RTK when available)
SLAM	EKF-SLAM (IIT Bombay, OzU). FastSLAM 2.0 moving to GraphSLAM (EUFS's own comparison found GraphSLAM better; Trinity switched). Cartographer (HWU Dubai).	GraphSLAM or factor graph 13, EKF 10, FastSLAM 3	EKF-SLAM first, GraphSLAM (GTSAM iSAM2) second
Planning	Delaunay mid-line, cone midpoints, ft-fsd-path-planning	Delaunay 4, race-line optimisation 4	Delaunay mid-line plus a speed profile
Control	Pure pursuit or Stanley, plus PID, everywhere	MPC 18	Pure pursuit and PID; MPC later
Compute	Jetson AGX Orin, or the IMechE InCarPC (i7-8700T, GTX 1060)	Intel industrial PC 18, Jetson Orin 15, AMD 7; several i7/i9 + RTX 4060-class desktops	Advantech MIO-5379 (Core Ultra 7 255H), no GPU; an Orin NX only if the camera needs the ZED SDK (Computer)

The 2026 Driverless Cup podium shows where the top teams are heading. TU Hamburg won with a MEMS lidar, three radars, no camera, and MPC. NTNU was second, with a 192-channel RoboSense lidar, two ZED X One cameras, graph SLAM with an EKF, and MPC. ETH was third; its 2025 entry lists a Hesai OT128, a Basler camera, GraphSLAM with iSAM2 and MPC. All of these are 60+ km/h cars. The ADS-DV has 5 kW, so the simple stack above loses little at its speeds.

Latency budget



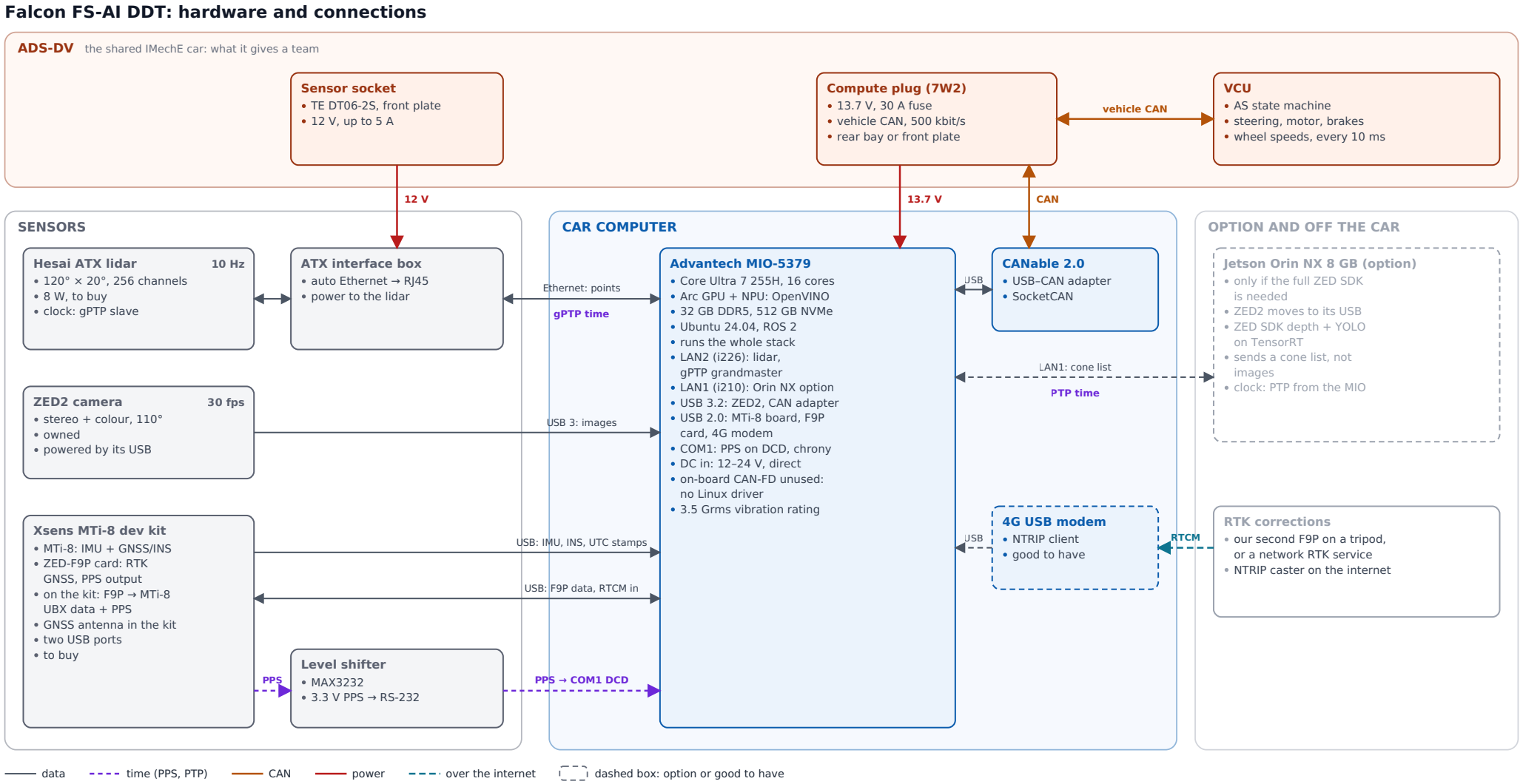
Milliseconds, to scale. Reference points from the papers: AMZ 2019 measured about 300 ms from cone detection to command; AMZ 2020 cut its perception-to-boundary pipeline to 112 ms; KIT19d kept every module cycle under 50 ms.

From a laser pulse leaving the lidar to the front wheels moving. The steering actuator figure is an assumption until we measure a step response on the car.

About 150 ms of software delay means a cone is 1.5 m closer than when it was scanned. Three things keep that harmless:

1. Every detection is placed in the map using the pose at its own timestamp, so delay shifts *when* we learn about a cone, not *where* we think it is.
2. The controller never waits for perception. It tracks the current path with a pose that is at most 10 ms old.
3. The look-ahead point is kept beyond the distance covered in the actuator delay: at least 2.5 m, growing to about 8 m at 10 m/s.

Hardware: what connects to what



Arrowheads at both ends: both directions. One clock for everything: the F9P's PPS sets the computer's clock (chrony), the computer passes it to the lidar (gPTP), and the MTi-8 takes the same PPS on the kit.
The ZED2 has no clock input: its frames are stamped on arrival in the computer, less a fixed delay found by calibration on the rig.

Figure 2. The car’s hardware and every connection. Dashed boxes are the Orin NX option and the good-to-have 4G modem.

One computer, the Advantech MIO-5379, runs the whole stack, and every sensor plugs straight into it. The car gives a team two connections: the 7W2 compute plug with 13.7 V (30 A fused) and the vehicle CAN at 500 kbit/s, and a TE DT06-2S socket on the sensor plate with 12 V, up to 5 A (IMechE compute document).

Device	Port on the MIO-5379	Carries	Clock
Hesai ATX, through its interface box	LAN2, Intel i226	points (UDP); gPTP both ways	follows the computer over gPTP and stamps each point at capture
ZED2	USB 3.2 Type-A	left and right images, the camera's IMU	none: stamped on arrival, less a fixed delay found on the rig
MTi-8 dev board	USB 2.0	IMU, orientation, INS position and velocity	follows the F9P's PPS on the kit; packets carry UTC
ZED-F9P daughter card	USB 2.0	F9P position, velocity and time out; RTCM corrections in	GNSS
F9P PPS, through a MAX3232	COM1, DCD line	one pulse a second	sets the computer's clock (chrony)
CANable 2.0 (USB–CAN)	USB 3.2 Type-A	vehicle CAN, through the 7W2 plug	each frame stamped on arrival by SocketCAN
4G USB modem (good to have)	USB 2.0 header	NTRIP corrections for the F9P	—
Jetson Orin NX (option)	LAN1, Intel i210	a cone list with colours and times	PTP from the computer
Power	DC in, 12–24 V	13.7 V from the 7W2 plug, no converter	

Ports: the MIO-5379 has two USB 3.2 Type-A, one USB-C 3.2, one USB4, two USB 2.0 Type-A and two USB 2.0 headers, so every device has its own port and the USB-C and USB4 stay free. COM1 comes out on a 20 cm D-sub cable that ships with the board (datasheet, p. 2).

The Xsens MTi-8 kit

- **Module and kit.** The MTi-8 itself is a module, not a board: 12.1 × 12.1 × 2.55 mm on a PLCC-28 footprint, 2.8–3.6 V, and only UART, SPI or I^2C, with no USB (MTi-8 leaflet). The development kit puts it in a socket on Xsens's dev board, which adds USB, and adds an RTK daughter card with a u-blox ZED-F9P, a barometer, an XBee socket and its own USB connector, plus an RTK antenna and a USB cable (Xsens shield board documentation).
- **Why this unit.** The kit's F9P already feeds the MTi-8 its position and its PPS, so the MTi-8's clock follows GNSS ("1 ppm (external)") and it fuses RTK with its IMU on board. Gyro in-run bias 6 °/h and noise 0.003 °/s/√Hz, accelerometer bias 40 µg, output up to 1 kHz (leaflet). The kit's F9P is the rover; our two F9Ps become the RTK base on a tripod and a spare.
- **Price.** USD 905.01 at DigiKey US (3 in stock), under the USD 1,000 limit for the IMU before shipping and duty; LKR 421,112 landed. Arrow lists USD 745.21 with no stock.
- **Mounting.** A socketed module can work loose under vibration: clamp it in its socket, or move it to a small soldered carrier later.
- **Ask Movella before ordering** whether the daughter card's USB reaches the F9P itself (for RTCM in and its time messages) and which header pin carries the F9P's PPS for the computer's clock.

The Hesai ATX

- 120° × 20°, 256 channels, 0.08° × 0.05° at the finest, 230 m on a 10% target, 8 W, 100 × 100 × 30 mm, 360 g, IP6K7 and IP6K9K (Hesai). Why one 120° lidar is enough for the tightest hairpin, and why the VLP-16 stays on the rig: the Lidar Coverage and Hairpins and Lidar Resolution and SLAM pages.
- About USD 1,540 for one unit from a Hong Kong seller on Alibaba, which lists a 10 Hz frame rate, 5.0 kg gross and a 31-day lead time. LKR 718,765 landed, with the shipping and the lidar's customs line estimated (Cost).

- Hesai's product page does not give the ATX's electrical interface. Its other automotive lidars use automotive Ethernet (1000BASE-T1), with an optional connection box that gives a standard Ethernet port and takes the power (QT128C2X manual). Figure 2 draws that box; ask the seller what comes with the unit.

One clock: GNSS time everywhere

Lidar de-skew and lidar–IMU fusion need the sensors' time stamps to agree to well under a millisecond. Everything is put on GNSS time in one chain:

1. **F9P → computer.** The F9P solves its own clock against the satellites and pulses once a second, within 30 ns RMS (data sheet, p. 4). A MAX3232 turns the 3.3 V pulse into RS-232 levels for COM1's DCD line. chrony takes it as a PPS reference, numbers the second from the F9P's time messages over USB, and steers the system clock.
2. **Computer → lidar.** `phc2sys` copies the system clock to the i226 port's hardware clock, and `ptp4l` runs there as an IEEE 802.1AS (gPTP) grandmaster: layer 2, peer-to-peer delay, `transportSpecific 1`, destination 01:80:C2:00:00:0E (ISC's ptp4l.conf). ISC Racing run the same lidar this way. Their notes say the ATX does not see a default 1588 end-to-end master, software time stamping leaves ±100–300 µs, and hardware time stamping is what reaches the lidar's locked state (ISC sync notes). Both MIO-5379 ports, the i226 and the i210, time-stamp in hardware. The Hesai driver is set to the lidar's capture time (`use_timestamp_type: 0`), not the host's arrival time.
3. **F9P → MTi-8.** On the kit the same pulse reaches the MTi-8, so its clock follows GNSS and its packets carry UTC. The Xsens ROS 2 driver uses that UTC time when `time_option` is 0, but does not check whether the UTC field is valid yet (driver source), so the stack starts only once the F9P has a fix.
4. **Camera.** The ZED2 has no clock input. Our zed-open-capture node stamps each frame on arrival, less a fixed delay measured once on the rig, for example by filming an LED driven by the PPS.
5. **CAN.** SocketCAN stamps each frame with the system clock on arrival.
6. **Orin NX option.** `ptp4l` on LAN1 with the Orin as slave; the Orin stamps each cone list with its image's time.

Without satellites, in a garage or under cover, the F9P keeps pulsing once a second but no longer on the top of the second (integration manual, p. 67). The lidar still follows the computer over gPTP, so the two stay together; set chrony to use the pulse only while the F9P reports a fix.

The car computer: Advantech MIO-5379

The MIO-5379C7P-R1A1U is a 3.5-inch industrial board with a Core Ultra 7 255H, the computer KIT raced (R6, 2026, 3/4). It is rated for 3.5 Grms vibration, runs on 12–24 V straight from the car, and has two Ethernet ports and four serial ports on the board (datasheet). The comparison with the other computers, its Linux support and how to set it up are on the DDT Compute Options page. One rule shapes the software: **every block runs on the CPU, the Arc graphics and the NPU.** No NVIDIA GPU is planned; the Orin NX below is an option for the camera only.

Block	On the MIO-5379	With the Orin NX option
Lidar cones, odometry EKF, cone SLAM, planner, controller, system	CPU. Geometry and small filters, a few milliseconds each.	unchanged
Cone colour	A patch of the left image at each lidar cone, named by an HSV rule or a small CNN on OpenVINO (Arc graphics or NPU). Fallback: full-image YOLO on OpenVINO; on a Core Ultra 7 155H, YOLO26n takes 5.8–9.1 ms and YOLO26s 9.3–16.7 ms per image (Ultralytics OpenVINO benchmarks), inside the 25 ms budget.	YOLO on TensorRT with ZED SDK depth on the Orin; it sends coloured cones
ZED2 driver	`zed-open-capture` in a small ROS 2 node of our own	the ZED SDK and its ROS 2 wrapper, on the Orin

Training the colour model and YOLO	on the team's RTX laptop	same

- **CAN.** The board's two CAN-FD ports have no Linux driver (Windows only, through Nuvoton's driver), so the car's CAN comes in through the CANable 2.0 on USB. Ask Advantech for a Linux driver before ordering.
- **Top teams run without a GPU.** TU Hamburg won the 2026 FSG Driverless Cup with a COM Express module carrying an i7-13800HRE, stated at 50 W (FSG page, DC26 results). KIT came 6th with the same Core Ultra 7 255H at 60 W (FSG page). Neither lists a camera or a GPU.
- **The shared car's InCarPC stays a fallback.** An OpenVINO model also runs on its i7-8700T, while TensorRT 10 no longer supports its GTX 1060.

The Orin NX option: zed-open-capture against the full ZED SDK

	`zed-open-capture`	ZED SDK
Runs on	any x86 or Arm computer; no CUDA	an NVIDIA GPU with CUDA: a Jetson, or a desktop card
Gives	the left and right images, the camera's IMU, and the factory calibration to rectify the images	all of that, plus stereo depth and point clouds on the GPU, positional tracking, object detection and the official ROS 2 wrapper (Stereolabs docs)
Job in our stack	the colour of each cone the lidar finds; range comes from the lidar	depth to check cones the lidar misses, and a full-image detector

Buy the Orin NX only if the rig shows that colour from zed-open-capture is not good enough. The set-up copies NTNU's (R2, 2026, 4/4): a Jetson Orin NX next to the Intel board. The ZED2 moves to the Orin's USB, and the Orin sends a cone list, not images, to the MIO-5379 over LAN1. The unit priced is Seeed's reComputer J4011 (Orin NX 8 GB, 128 GB SSD, no Super Mode) at USD 999 (Seeed), LKR 448,341 landed; its listing names no power adapter. The Orin NX 8 GB module runs at 10–20 W (NVIDIA). The Jetson Nano we already have can try the same split on the bench first (DDT Compute Options page).

Power

Build	At the car's 13.7 V
MIO-5379 with the CPU capped at 45 W; the ZED2, MTi-8 kit, CANable and 4G modem on its USB; the ATX on the sensor socket	76 W, 5.6 A (19% of the 30 A fuse)
The same, plus the Orin NX option	96 W, 7.0 A
The ATX alone, on the sensor socket	8 W, 0.7 A of its 5 A
Stated by other teams, processing units only	
TU Hamburg (1st, 2026) · KIT (6th, 2026)	50 W · 60 W
Median of the ten FSG teams that pair an Intel CPU with an NVIDIA GPU	220 W

Our estimate, from the Power budget tab of the cost workbook: the CPU at the 45 W cap we set in the BIOS (the 255H has 28 W base power and 115 W maximum turbo, Intel); the board, RAM, SSD and fan at 10 W and the board's regulators at 90% (both assumptions); the ZED2 1.9 W; the MTi-8 kit and CANable 1.5 W and the modem 3 W (assumptions); the ATX 8 W (Hesai); the Orin NX 8 GB at its 20 W top mode, carrier and fan not included. The team figures are from their FSG pages, collected in the workbook's FSG x86 teams tab. The computer must also fit about 280 × 200 × 200 mm at the rear and needs IMechE approval in the ASF (T1.1.6).

Cost, landed in Sri Lanka

Landed means goods + shipping + Sri Lankan duties, at the Customs rates for 2–8 November 2026: USD 337.1431, EUR 377.4991, CHF 410.5493 (Customs notice RE/44/2026). Duties are the levies for the HS code (CID, PAL, CESS) on the CIF value, then SSCL 2.5% and VAT 18% on 1.1 × CIF plus those levies. The shipping on imports and the lidar's HS code are estimates. Every line, with its listing link and the formulas, is in the cost workbook `ddt_essential_costs.xlsx`; prices were checked on 6 November 2026.

Item	Listing	Landed LKR
Essential: the car		
Hesai ATX lidar	about USD 1,540, Alibaba; shipping and HS code estimated	718,765
Advantech MIO-5379C7P-R1A1U, cooler included	USD 1,306 + USD 22.97 shipping, Advantech eStore	576,084
2 × Crucial 16 GB DDR5-5600 SO-DIMM	LKR 70,500 each, Chama	141,000
512 GB Gen 4 NVMe SSD (the listing names no brand; check it is M.2 2280)	LKR 30,000, Chama	30,000
Xsens MTi-8 development kit (MTI-8-5A-DK), with its ZED-F9P card and antenna	USD 905.01, DigiKey US	421,112
CANable 2.0 USB–CAN adapter	USD 35, Openlight Labs	22,463
Norcomp 7W2 compute plug for the car	CHF 10.49, RS	13,430
Essential: the HiL bench and the test rig		
PEAK PCAN-USB, for the Windows PC that runs CarMaker	USD 256, Grid Connect	118,262
Test rig: hand truck, aluminium profile and brackets, 12 V 18 Ah battery and charger, encoder, ESP32, CAN transceiver, Deutsch connectors	nine items, Daraz.lk (listed in the workbook)	74,225
Essential total		**2,115,342**
Good to have		
Huawei E3372-325 4G USB modem. Why: RTK at the event needs corrections over the internet, if IMechE agrees in the ASF.	LKR 40,378 delivered, desertcart.lk	40,378
Essential + good to have		**2,155,720**
Option, only if the full ZED SDK is needed		
Seeed reComputer J4011, Jetson Orin NX 8 GB, 128 GB SSD	USD 999, Seeed	448,341

- **Owned, no cost:** the ZED2; two ZED-F9P with antennas (the RTK base and a spare); the VLP-16 (the rig's lidar until the ATX arrives); the RTX laptop for training; the Jetson Nano and the borrowed AGX Xavier.
- **Needed, not priced yet:** the ATX's connection box and cable if they do not come with it; a MAX3232 module and DB9 lead for the PPS; the MIO-5379's 2 × 2 power lead, the CAN leads and a fuse; the computer's enclosure with a fan, vibration isolators, and the lidar bracket.
- Cones are made in-house, and the 1:10 car for stage 3 is not in the workbook.

Two vehicle profiles

A profile is one parameter file plus one launch argument. Code never branches on the vehicle name. It reads numbers from the profile, and the profile chooses which drivers the vehicle layer starts.

	ADS-DV	1:10
Used in	stage 1, 2a, 2b and the event	stage 3, and its reference runs in stage 1
Commands go out through	CAN bridge → AI2VCU frames	DDS over Wi-Fi → Raspberry Pi 5 → VESC and steering servo
Wheel speeds and steering angle	VCU2AI frames. On the rig, an ESP32 sends them from an encoder. In simulation, the sim bridge computes them from the vehicle state.	VESC motor speed; the servo command
Geometry	wheelbase 1.530 m, track 1.200 m (specification)	measured on the chosen chassis
Limits	steering about ±28° in the CarMaker model (measure it); 55.7 Nm motor through 3.5:1	servo travel and VESC current, measured
Sensor frames	from the official CAD (sensor plate)	MTi-8 kit position on the RC car
Track	full size	cones scaled by the wheelbase ratio, so corners are as tight relative to the car
Controller gains	tuned on CarMaker, refitted from test-day step responses	tuned in stage 1 on the 1:10 model

Test stages

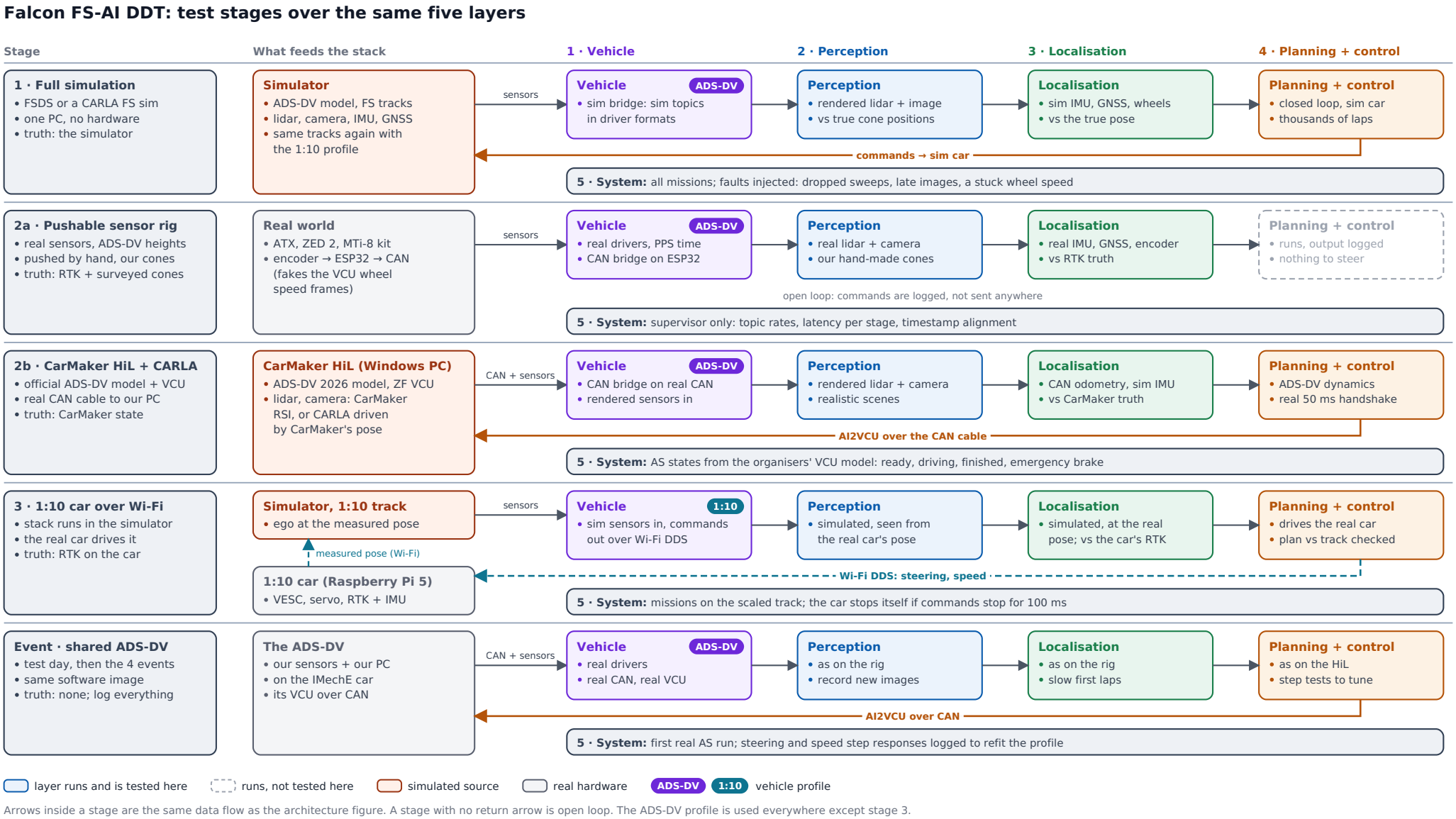


Figure 3. Each stage drawn on the same layers as figure 1. Dashed means the layer runs but is not tested there.

Stage	What is real	What it proves	Metrics
1 · Full simulation FSDS or a CARLA-based FS simulator	nothing; one PC	The whole stack, end to end, on every mission, with faults injected on purpose.	Cones hit, missions not finished, lap time, cross-track error; perception recall by range; pose error against the simulator's truth.
2a · Pushable rig	sensors, drivers, time sync, cones	Perception and localisation on our real sensors, at the ADS-DV's mounting heights.	Detection recall and false cones by range; colour accuracy; cone position error against surveyed positions; pose error against RTK; latency of each block.
2b · CarMaker HiL + CARLA	the CAN cable, our CAN bridge, the organisers' VCU model	CAN handshake and AS states; localisation against exact truth in realistic scenes; control on the official vehicle model.	Handshake reply time; AS state transitions; trajectory error (ATE/RPE) against CarMaker's state; map error against the true cones; lap time.
3 · 1:10 car over Wi-Fi	the actuators, tyres and lag of a real car, plus a Wi-Fi link	The planner's and controller's outputs hold up on a real plant, not just the model.	Plan vs track error, prediction error, shadow-model gap, latency. Details.
Event shared ADS-DV	everything	The first real AS run.	Steering and speed step responses to refit the ADS-DV profile; images of the real cones recorded for training.

Stage 1: FSDS or a CARLA-based simulator

- **FSDS:**
  - The v2.2.0 documentation lists lidar, camera, IMU, GPS and a ground-speed sensor.
  - The repository has a ROS 2 bridge (`fsds_ros2_bridge`) and was last pushed in September 2025.
  - There is no wheel-speed sensor, so the vehicle layer's sim bridge computes the four wheel speeds and the steering angle from the vehicle state, in VCU2AI units. Localisation then sees the same messages as on the car.
- **FSDS also ships an ADS-DV vehicle** (contributed by Imperial). Its commands are normalised throttle, steering and brake, so the sim bridge converts our torque and brake requests.
- **CARLA:** HWU Dubai's 2025 stack was built in CARLA, but with a stock CARLA car, and it scored no dynamic points. If we use CARLA, give its car the ADS-DV numbers from the vehicle model table and match CarMaker's step responses.
- **Run every track twice:** once with the ADS-DV profile, and once with the 1:10 profile. The 1:10 runs are the reference that stage 3 is checked against.
- **Inject faults:** dropped lidar sweeps, late images, a stuck wheel speed. These are the reliability failures that end a trackdrive.

Stage 2a: pushable rig

- **Mounting:** the real sensors sit at the ADS-DV's mounting heights, taken from the CAD.
- **Fake wheel speeds:** an encoder on a wheel drives an ESP32 that sends VCU2AI wheel-speed frames on CAN. The CAN bridge and the odometry EKF run unchanged.
- **Ground truth:** the RTK base sits on a tripod and the rover rides on the rig. Survey every cone with the rover before the run.
- **Open loop:** the planner runs but nothing moves. Check its path against the surveyed track.
- **Lidar test:** the VLP-16 runs on the rig until the ATX arrives. Then push the rig through a 9 m hairpin and compare the inside-edge cones in the map with their surveyed positions, to confirm that the ATX alone holds the map through the turn (Lidar Coverage and Hairpins page).

- **Time test:** check that the lidar reports gPTP locked, and that IMU, lidar and CAN time stamps agree with the computer's clock (Time).
- **Computer test:** run the full stack on the MIO-5379 and check every block's rate and latency, and the colour accuracy from `zed-open-capture`. If the colour falls short, the Orin NX option is the fix (Orin NX).

Stage 2b: CarMaker HiL + CARLA, and whether CarMaker alone is enough

- **What the official repository already has:**
  - The non-HiL 2026 vehicle defines a Lidar RSI and two Camera RSI sensors (RGB and depth).
  - The bundled ROS 2 interface (CMRosIF) publishes them as `/pointcloud`, `/front_camera_rgb/image_raw` and `/front_camera_depth/image_raw`.
  - So CarMaker can render sensors by itself.
- **Limits:**
  - That lidar is 120° × 10° with 300 × 50 beams. The ATX matches its width but not its 20° height, so we write a beam table for the ATX.
  - The ROS path is the Linux, CarMaker Office route, kept from the upstream project. The supported HiL workflow is Windows over CAN.
  - The HiL vehicle file has these sensors switched off.
  - RSI sensors render on the GPU.
- **Recommendation:** try CarMaker alone first: one simulator, exact truth, nothing to synchronise. Fall back to CARLA as a renderer that follows CarMaker's pose if the licence or the GPU rules out RSI.
- **Localisation metrics:** ATE and RPE against CarMaker's vehicle state, and map error against the track's true cone positions. Repeat with sensor noise and lighting changes.

What the HiL and CarMaker are

CarMaker is IPG Automotive's commercial vehicle-dynamics simulator: a detailed model of a car (chassis, suspension, steering, powertrain, brakes, tyres) driving on a virtual track. IPG gave every FS-AI team a free licence in 2025 (IPG).

HiL (hardware-in-the-loop) means our real computer, running our real software, is wired by a real CAN cable to a PC running CarMaker. CarMaker plays the ADS-DV: the organisers' 2026 vehicle model plus the VCU and AS state machine, talking the 2026 CAN protocol at 500 kbit/s. Our computer cannot tell it from the car, so the CAN bridge, the 50 ms handshake, mission selection, the planner and the controller are tested with real CAN timing. This is IMechE's own set-up, published as FS-AI_IMechE_ADS-DV_HiL (last updated 25 April 2026). Its README says it runs only on the Windows version of CarMaker Extended 14.1.1.

Two details matter for us. First, the HiL configuration is written for a PEAK PCAN-USB adapter on the Windows PC (`Slot.200.Module = PCAN-USB`, channel `PCAN_USBVCU`), which is why the PEAK is in the essential list; our own computer needs its own CAN port, the CANable. Second, in the HiL vehicle file the simulated cameras and lidar are switched off (sensors 3–5, `Active = 0`), so the HiL gives our computer only what the real car's CAN carries. In stage 2b the cones come from rendered sensors instead: CarMaker's own RSI sensors, or CARLA (above).

Where the files are

Each row links to the file in IMechE's official FS-AI repositories on GitHub.

What	File or folder
HiL project, README and user guide	`FS-AI_IMechE_ADS-DV_HiL` repository `doc/FSAI_UsersGuide.md`
ADS-DV 2026 vehicle data set (HiL variant, and the variant with the simulated sensors on)	`Data/Vehicle/FS-AI_IMechE_ADS-DV_2026_HiL` `Data/Vehicle/FS-AI_IMechE_ADS-DV_2026`

Tyre (“Generated – not validated”)	`Data/Tire/RT_185_50R13`
Suspension kinematics	`Data/Chassis/`
VCU, AS state machine and vehicle model sources (ZF)	`src/ZF_src/`
CAN set-up for the PEAK channel	`Data/Config/FSAI_PCAN.iocfg` `Data/Config/FSAI_PCAN.cancfg`
CAN database (HiL, without the PCAN-GPS messages; and the full car)	`CANdb/ADS-DV_2026_CAN1_no_GPS.dbc` `ADS-DV_2026_CAN1.dbc`
Cone 3D models and the cone placement script	`Movie/TrafficCones/` `doc/Traffic Cone Placement Script.pdf`
Vehicle CAD (STEP and Fusion 360)	`FS-AI_ADS-DV_CAD_2026.step`
Specifications and dimensions; software interface	`FS-AI ADS-DV Specifications & Dimensions.md` `FS-AI ADS-DV Software Interface Specification.md`
Fitting our own computer: power, CAN, size	`ADS-DV_Reference_Compute_Installation.md` `CAD/ADS-DV_Rear_Compute_Installation.stp`
EUFS bicycle-model parameters fitted to the ADS-DV	EUFS `vehicle_models/config/DynamicBicycle` (GitLab)

Stage 3 in detail: the simulated stack drives the real 1:10 car

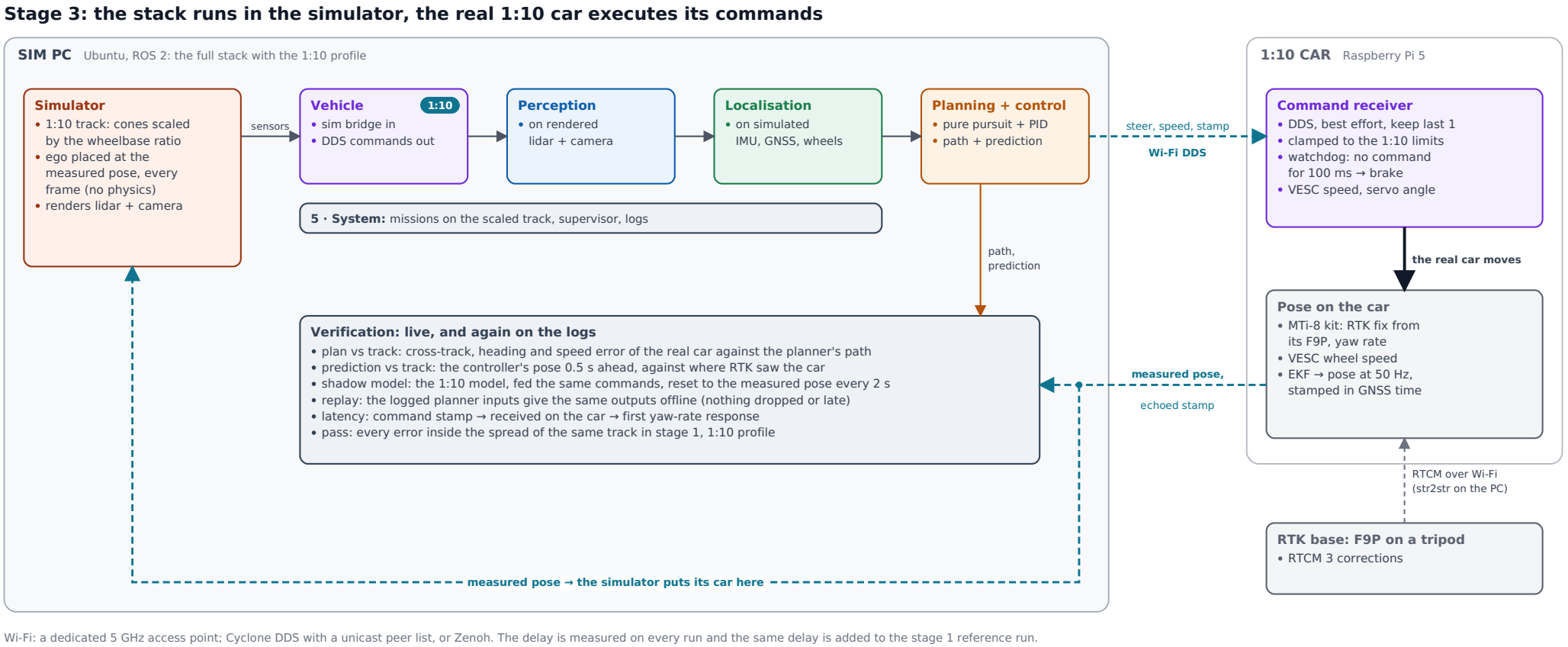


Figure 4. Stage 3: Wi-Fi links dashed in teal.

- **Why the measured pose goes back into the simulator:**
  - Without it, the simulated car and the real car drift apart within seconds, and the real car just copies commands.
  - With it, the simulator draws the world around where the real car actually is. The loop is then closed through real tyres, servo lag and Wi-Fi delay.
- **How to check that planning is "correct as in simulation"** (the verification box):
  - **Plan vs track:** cross-track, heading and speed error of the real car against the planner's path.
  - **Prediction vs track:** the pose the controller expected 0.5 s ahead, against where RTK saw the car.
  - **Shadow model:** the 1:10 vehicle model, fed the same commands and reset to the measured pose every 2 s. The gap measures how wrong the model is. Use it to tune the 1:10 profile first.
  - **Replay:** feed the logged planner inputs to the planner offline. The outputs must match the live ones, which proves nothing was dropped or late.
  - **Latency:** command time stamp → received on the car → first yaw-rate response.
  - **Pass when:** every error lies inside the spread of the same track in stage 1 with the 1:10 profile, run with the same delay added.
- **Wi-Fi DDS settings:**
  - A dedicated 5 GHz access point.
  - Cyclone DDS with a unicast peer list, or Zenoh, because default DDS discovery relies on multicast, which Wi-Fi handles badly.
  - Commands as best effort, keep last 1: a late command is worse than none.
- **Watchdog on the car:** no command for 100 ms → brake. This mirrors the ADS-DV, where a late handshake fires the emergency brake.

Later option, same 1:10 profile: run the whole stack on the Pi, with virtual cones around the RTK pose, to test it on small hardware without Wi-Fi in the loop.

Weak points of teams without their own car, and what covers each

From the teams above. The four teams without their own car that finished the trackdrive are IIT Bombay, Trinity, Ain Shams and Bath. We copy the two physical tools they used: IIT Bombay's scaled robot becomes our 1:10 car, and Trinity's CAN-emulating "Rigby" becomes our ESP32 on the rig plus the official HiL.

Weak point	Seen in	What covers it	Stage
Perception that worked in simulation failed on the real hardware	UWE-AI: simulation confidence "did not carry over"; perception failures on the supplied hardware	Real sensors and real cones on the rig, months before the event	2a
The simulated car was not the ADS-DV	HWU Dubai: a stock CARLA car	The organisers' CarMaker ADS-DV model; any other simulator's car tuned to its step responses	1 2b
CAN interface written or debugged at the event; a late handshake fires the emergency brake	EUFS advice to spend ADS-DV time testing, not writing interface code; Trinity's Rigby still being debugged	Our CAN bridge against the organisers' VCU model over a real CAN cable	2b
Control never met real lag, slip and noise	UWE-AI: missing feedback control	A real 1:10 plant in the loop, with Wi-Fi delay measured	3
Little time on the shared car	Ain Shams (little time on the real car at the event); Bath (ran the organisers' car)	Arrive with every layer tested; a written test-day plan: step tests, slow laps, image recording	event

Few real images of the competition cones	IIT Bombay took about 3,500 cone images on the ADS-DV for training	Rig images of our cones made to the official drawings; more recorded on test day	2a event
One fault ends a 10-lap trackdrive	every team	Thousands of simulated laps with injected faults; the supervisor	1
Confidence from simulation only	OzU: CarMaker with the ADS-DV model, no dynamic points in 2025 or 2026	Every layer meets something real before the event except the ADS-DV's own dynamics, which come from the official model	all

Sensors, the IMU and GPS

Sensor	Status	Use
Hesai ATX	**to buy**	Cone positions: 120° × 20°, 256 channels, so cones are found further out than with the VLP-16. One 120° lidar gets through the tightest hairpin when the planner works on the map (Lidar Coverage and Hairpins page). The only lidar on the car.
Velodyne VLP-16	owned	The rig's lidar until the ATX arrives. Not on the car.
ZED2	owned; the shared car also comes with one	Cone colour (why the car needs it). Ours lives on the rig; at the event, use the car's or fit ours. It runs on `zed-open-capture`; the ZED SDK only with the Orin NX option (Orin NX).
Xsens MTi-8 kit	**to buy**	Yaw rate and acceleration for the 100 Hz EKF and lidar de-skew, on GNSS time, with RTK fused on board. The car's own PCAN-GPS gyro is a consumer L3GD20 at 50 ms. See the comparison below.
ZED-F9P on the MTi-8 kit	comes with the kit	The rover: velocity, PPS time for every clock, RTK at the event over NTRIP if allowed.
u-blox ZED-F9P ×2	owned	The RTK base on a tripod for ground truth on the rig, and a spare.
2D lidars ×2, RealSense	owned	Not needed in this plan.

Which IMU: the Xsens MTi-8 kit

The IMU budget is at most USD 1,000. Under it, the MTi-8 kit is the only unit whose clock follows GNSS on its own, so its samples share a clock with the lidar points with no estimating in software.

Unit	What it is	How its clock is synchronised	Price found	Used by
Xsens MTi-8 kit (chosen)	IMU and GNSS/INS module on a USB dev board, with a ZED-F9P card and antenna	the kit's F9P feeds it the PPS: clock drift "1 ppm (external)", UTC on every packet (leaflet)	USD 905.01, DigiKey US; LKR 421,112 landed	no FSG team names it
Xsens MTi-3 kit	IMU module on a USB dev board	none: its sync input only asks for the latest sample (SendLatest), so samples are stamped on arrival or matched to the computer's clock in software	EUR 357.57, Farnell	—
Xsens MTi-630 kit	boxed AHRS, 4.5–24 V	sync-in and sync-out pins that can take the PPS, but the ROS 2 driver ignores them, so it needs our own code	USD 1,062.51, Farnell: over the limit	its sibling with GNSS, the MTi-680G, is in 5 FSG cars

SBG Ellipse-D	boxed INS, dual-antenna RTK GNSS	its own GNSS	INR 800,000 ex tax (the only public price), about LKR 3.76 M landed before antennas	Zurich UAS Racing (with a VLP-16, ZED 2i and AGX Xavier), Ingolstadt, Eindhoven

The Ellipse-D's two antennas would give heading at a standstill on the start line, without a magnetometer next to the motor (SBG purchasing guide), but it costs several times the whole essential list. Ask SBG for a student quote or sponsorship; two of our F9Ps can also give dual-antenna heading as moving base and rover.

GPS: the kit's F9P is the rover

The ZED-F9P on the MTi-8 kit is the rover; our two F9Ps, which came with their antennas, are the base and a spare. The F9P is a multi-band receiver with centimetre RTK, velocity without corrections, moving-base heading with a second F9P, and the PPS output that sets every clock on the car. The Piksi Multi used by TUW and MUR reached end of life on 15 March 2024.

RTK at the event over NTRIP. Rule T4.5.2 allows (D)GPS and says only that there is "no space to securely build up base stations on the competition site". Corrections can come from elsewhere over the mobile network: the car's computer runs an NTRIP client (the ROS 2 `ntrip_client` package, or RTKLIB `str2str`) and passes the RTCM stream to the kit's F9P over USB. The F9P needs RTCM 3 MSM4 or MSM7 observations plus a 1005/1006 base position (integration manual).

Source of corrections	Cost	Notes
Commercial network RTK (a virtual base computed next to the car)	RTKdata: USD 40 a month, 30-day free trial. TopNET Live: UK short licences from GBP 100 + VAT.	Baseline length stops mattering. RTKdata streams RTCM 3.2 MSM4. TopNET Live uses Ordnance Survey reference data.
Free community bases (RTK2go)	free	On 19 October 2026 four bases were within 50 km of Silverstone: Middle Barton 29 km, Flitton 39 km, Rugby 42 km, Bedford 45 km. Volunteer-run, so any of them can disappear without notice.
Our own base	the second F9P we own, plus a laptop and internet where it sits	u-blox: a single base should be within 50 km of the rover, and longer baselines slow the fix. **A base in Sri Lanka cannot work.** In the UK it could sit at the team's accommodation, surveyed in, streaming through RTK2go or our own `str2str` caster.

The catches. Wireless at dynamic events "may be limited" and is not guaranteed (T4.5.1), so the stack must run without RTK. Rule D1.3.1 bars the team's monitoring device from sending data to the car; an internet corrections stream is not that device, but it is outside data reaching the car, so declare it in the ASF and get IMechE's agreement first. RTK gives the car's position, not the cones: its value is a cleaner pose and heading for the EKF and SLAM, and ground truth on the test day. Good to have, not essential.

Fitting our kit to a shared car. Shared ADS-DVs come with the InCarPC and a ZED2. Our own computer and sensors need IMechE approval, requested in the ASF (rules T1.1.6 and T4.4.3) and designed against the official CAD. Shared cars may not use the low zone ahead of the front tyres (T1.3.4). The 2026 CarMaker model puts the sensor plate about 0.39 m behind the front axle, with the lidar at 0.57 m and the camera at 0.62 m above the ground, pitched 15° down; the rig copies those heights. Sensors get 12 V (11.5–14.5 V), up to 5 A, on a TE DT06-2S socket at the front, which feeds the ATX. A team computer plugs into a 7W2 combo D-sub carrying 13.7 V nominal, 30 A fused and shared with the sensor plate, and the vehicle CAN at 500 kbit/s, already terminated. At the rear the computer must fit about 280 × 200 × 200 mm and weigh at most 8 kg with its plate; it can also go on the front sensor plate, which has the same connector (IMechE compute document)).

Cones: make our own

The official cones are not costed: we make them. The FS-AI cone specification (adapted from the FSG handbook) names these WEMAS cones:

Cone	Base × base × height	Mass	Colour	WEMAS part
Small blue (left edge)	228 × 228 × 325 mm	0.45 kg	blue, one white stripe	`400.000043.00.00`
Small yellow (right edge)	228 × 228 × 325 mm	0.45 kg	yellow, one black stripe	`400.000013.01.10`
Small orange (entry and exit lanes)	228 × 228 × 325 mm	0.45 kg	orange, one white stripe	`400.000013.00.00`
Big orange (start, finish, timing lines)	285 × 285 × 505 mm	1.05 kg	orange, two white stripes	`307.610500.00.00`

Stripe positions are not in the specification. The organisers' own 3D models in the HiL repository put the small cone's stripe 137–222 mm above the ground, and the big cone's two stripes at 138–223 mm and 308–393 mm. Those models are slightly off the specification in size (a 288 mm base, 320 mm tall), so build to the table and take only the stripe heights from the models.

Drawings to work from: the official models, `TrafficCone_Small_Blue.obj` and the yellow, orange and large ones next to it (OBJ files open in Blender or FreeCAD). Other teams publish their simulator meshes too, for example Technion's `TechnionSim/eufs_tracks/meshes`.

What matters for our sensors: height and base, because the lidar cone detector checks size and point count and tells a big orange cone by its height; colour and stripe, because the camera detector learns them; and a matte plastic finish like the real cones. Mass only changes how a cone falls when hit. No full-size cones were listed on Daraz.lk on 19 October (only toy cones).

Why the car needs colour: the orange cones

Blue and yellow only tell left from right, and after the first lap the map does that too. Orange cones carry what nothing else gives the car: where each run starts and ends, and where to leave the track. The cone specification puts big orange cones "before and after start, finish and timekeeping lines" and marks exit and entry lanes with small orange cones (DE6.3.1, p. 1). Every mission ends at one of those lines (FS-AI 2026 rules, PDF pages below):

Mission	What the car must do at the orange cones	Rule
Acceleration	Stop within 100 m after the finish line, inside the marked exit lane.	D5.3.6, p. 32
Skidpad	Count four laps at the line between the circle centres (two right, two left), leave through the exit lane and stop within 25 m after the timekeeping line. A wrong number of laps is a DNF.	D4.1.6, p. 30; D4.3.5–4.3.6, p. 31; D9.1.11, p. 35
Autocross	Stop within 30 m after the finish line.	D6.3.7, p. 33
Trackdrive	Count ten laps itself, since no last-lap signal is sent, then stop within 30 m after the finish line.	D8.2.7–8.2.8, p. 34

- **The lidar finds big orange cones by height, close in.** They are 505 mm tall against 325 mm for the others, so the lidar detector flags them as a check on the camera. It needs several rings on the cone, which a denser lidar gives from further away.
- **Only the camera tells a small orange cone from a blue one.** They are the same size, and both have a white stripe; only the yellow has a black one. Lidar brightness, which some teams use to tell blue from yellow (What prior teams fitted), cannot separate them.
- **How the stack uses it.** The camera detector has four classes: blue, yellow, orange and big orange; the FSOCO dataset labels small and large orange cones separately. SLAM keeps each cone's colour in the map. The path planner finds the pair of big orange cones on the line and flags each time the pose crosses it, and the behaviour tree counts the crossings and sets the stop (figure 1).

Vehicle model for simulation

The ADS-DV can be modelled closely from published sources. The best one is the organisers' own CarMaker model in the official HiL repository. It is the same model their VCU emulation drives over a real CAN cable, so a controller tuned on it is tuned on the car the organisers simulate. The other sources fill in the geometry, the steering actuator and tyre numbers fitted to test data. Links to every file are in Where the files are.

Source	What it gives
IMechE HiL, IPG CarMaker 14.1.1 (updated 25 April 2026)	Complete 2026 vehicle data set: rigid body, suspension kinematics, springs, dampers, anti-roll bars, steering map, single-motor BEV powertrain with torque map, hydraulic brakes, aero, tyre, and sensor positions. Also the VCU and AS state machine (ZF model sources), tracks for all events, and the 2026 CAN database.
Official CAD, 2026	Geometry for meshes, sensor mounts and the sensor envelope. Released "for sensor and/or compute hardware installation and vehicle dynamics modelling", and must not be re-hosted publicly.
Specifications & Dimensions (April 2026)	Wheelbase 1530 mm, track 1200 mm, 5 kW, Saietta 119R-68 motor at 55.7 Nm, 3.5:1 belt, open differential, Toyo R888R 225/45 R13 tyres
EUFS dynamic bicycle model	Parameters fitted to ADS-DV test data (`ads-dv-exp`, `ads-dv-tuned`): yaw inertia about 150 kg·m², 49% front weight, tyre and drag coefficients
Microsteer EPAS datasheet	Steering actuator: 37 Nm, 860°/s no-load column speed (EPAS200M19 datasheet)

How fast the car is

The car is slow by Formula Student standards, which matters for every later choice. The 2026 specification gives 5 kW maximum electrical power, rear-wheel drive through an open differential, a 1.53 m wheelbase, and EUFS's CAN bridge uses 300 kg and a 0.253 m wheel radius. From those numbers, our estimate is about 2.5 m/s² off the line, power-limited above roughly 6.5 m/s, at most about 15 m/s at the end of the 75 m acceleration run, and 4–10 m/s on a track with 9 m hairpins. Treat these as estimates until we have driven it.

Key parameters, and where each goes in CARLA

Parameter	Value (CarMaker 2026 unless noted)	CARLA field
Mass	body 279 kg + 4 × (5.3 kg carrier + 6.2 kg wheel) = 325 kg	`VehiclePhysicsControl.mass`
Centre of mass	0.765 m ahead of the rear axle (mid-wheelbase), 0.15 m high ("lowered")	`center_of_mass`
Body inertia	Ixx 11.98, Iyy 53.17, Izz 51.12 kg·m² (EUFS fit: Izz about 150 for the whole car)	set in the Unreal blueprint
Wheelbase, track	1.530 m; 1.157 m in the model against 1.200 m in the spec	wheel positions in the mesh
Wheel	radius 0.2575 m (the CAD shows Ø515 mm), 6.2 kg, Iyy 0.476 kg·m²	`WheelPhysicsControl.radius`
Steering	linear road-wheel map to ±28°. The HiL VCU reports ±30°, earlier notes say 21–24°. Measure it.	`max_steer_angle`, `steering_curve`
Motor	55.7 Nm at 0 rpm, 42.9 at 1050, 28.6 at 1750, 14.3 at 2450, 2.9 at 3150, near 0 at 4000 rpm; 3.5:1 to the rear axle, no gearbox	`torque_curve`, `max_rpm`, `final_ratio`, one forward gear
Brakes	hydraulic; 4.0 / 3.5 Nm per bar front / rear; 50 bar master cylinder at full request; 80 ms build-up	`max_brake_torque`
Aero	drag coefficient 0.467 on 0.49 m², no downforce	`drag_coefficient`

Suspension	springs 6800 N/m front and rear; damper curves; anti-roll bar 1000 (CarMaker units); kinematics in `.skc` files	wheel suspension settings in the blueprint
Tyre	CarMaker uses a 185/50 R13 set marked "Generated – not validated", while the car runs 225/45 R13 Toyo R888R	`tire_friction`, `lat_stiff_value`, `long_stiff_value`

Gaps. The real tyre data, the steering actuator's rate and lag on the car, the VCU's torque and speed caps, and the sensor timing. The CARLA vehicle model is also cruder than CarMaker's: it has no suspension kinematics and only a simple tyre. **Recommendation:** use the CarMaker HiL as the reference model for control. If stage 1 uses FSDS or CARLA, tune its vehicle to match CarMaker step responses (steering step, full-throttle run, braking from 10 m/s). At the event test day, log steering and speed step responses and refit the tyre and steering numbers from those logs.

What prior teams fitted

Stacks described in papers and theses, in rough order of how much they tell us. Rates and results are as the authors report them.

Team, car	Perception sensors	Motion sensors	Estimation and mapping	Planning, control	Compute
AMZ *flüela*, ETH Zurich. Won FSG 2017. Valls 2018	Velodyne VLP-16 Puck; self-built stereo camera	INS with consumer GPS, "no differential GPS or RTK"; optical ground speed sensor; wheel encoders	EKF velocity estimate at 200 Hz with delay compensation and outlier rejection; FastSLAM; map frozen after loop closure	Plans on the map once it is closed	Industrial PC, GPU slave PC, real-time ECU
AMZ 2018–19. First overall at every event entered. Gosala 2018, Kabzan 2019	3D lidar and cameras as two independent pipelines; lidar colour from intensity only to 4.5 m	Dual-antenna INS (heading 0.08°); ground speed sensor; four wheel speeds	Slip-aware EKF, 200 Hz; FastSLAM 2.0 fed by lidar cones at 10 Hz and camera cones at 5 Hz; mapper 15 Hz	Lap 1: mid-line re-planned every 50 ms, pure pursuit. Later: MPC, 50 ms step, 2 s horizon	Master PC, slave PC, ECU
AMZ 2020 Andresen 2020	Lidar and camera	As above; RTK only as ground truth	Local map then GraphSLAM; mapping speed raised from 3 m/s to 12 m/s	—	Pipeline 112 ms
KIT19d, KA-RaceIng. Won 3 of 4 dynamic events at FSG 2019. Nekkah 2020	Several lidars; camera with MobileNetV2	—	Every module cycle under 50 ms	MPC, 20 ms step, 65 steps	Intel i7-9700K, 32 GB
KIT21d. "The stack that won FSD". Alvarez 2022	Hesai Pandar40P at 10 Hz, sweep de-skewed with odometry	Odometry and acceleration	GraphSLAM, optimised in the background	Minimum-curvature race line; linear time-varying MPC	—
IIT Bombay Racing. FS-AI DDT, on the ADS-DV. Rampuria 2024	Velodyne VLP-16; two ZED2i	SBG Ellipse-N GNSS/INS; ADS-DV CAN	SLAM on cone landmarks (MRPT)	Delaunay waypoints; pure pursuit	Jetson AGX Orin
EUFS, Edinburgh. FS-AI, on the ADS-DV. eufs_sim, ros_can, Newlands 2024	VLP-16 and ZED on a front sensor plate	SBG IMU; GPS; wheel speeds from CAN via a 100 Hz bridge	FastSLAM; a 2024 project compares it with GraphSLAM	—	—

QUT Motorsport Bradford 2023	Velodyne lidar; ZED2i	SBG INS with GNSS at 5 Hz; wheel encoders; steering	EKF at 50 Hz on IMU and wheel speed, feeding EKF landmark SLAM. Adding it improved map accuracy 65 % and pose tracking 81 %	Pure pursuit; Nav2	—
Elefant Racing, Bayreuth. FS East design award. Friedrich 2024	Two Basler cameras near, a stereo camera far; no lidar that season	Dual-antenna RTK with corrections over a dual-SIM cellular router; Kistler optical ground speed sensor; deliberately no wheel sensors	GNSS when available, ground speed sensor when not	—	Ryzen 3700X, RTX 2070 Super
Universidade de Vigo Fontán-Costas 2025	ZED 2i only; depth error under 0.5 m to 7 m	ZED 2i internal IMU at 400 Hz; ZED-F9P RTK at 10 Hz with a fixed base; rear wheel speeds	One EKF on a kinematic bicycle, stepped at 0.1 s to match the RTK rate	Pure pursuit, speed-dependent look-ahead	Jetson Orin NX
Zurich UAS Racing Finn 2025	Velodyne VLP-16; ZED 2i	SBG Ellipse-D dual-antenna INS	—	—	Jetson AGX Xavier
BIT-FSD, Beijing Tian 2020	Lidar; camera	GPS-INS	Lidar odometry at 10 Hz blended with GPS-INS in a complementary filter	—	—
FST Lisboa Lopes 2021, Morgado	Velodyne VLP-16	—	Landmark SLAM thesis	—	—
TUW Racing, Vienna Zeilinger 2017	ZED stereo	Piksi Multi differential GPS with the team's own two beacons	—	—	—
MIT / Delft Strobel 2020	Basler stereo and mono; YOLOv3 plus keypoints	—	—	—	Jetson Xavier

Stacks known mainly from their code. The entries are the drivers, libraries and controllers found in each repository, and each team name opens its folder. The hardware listed for HWU Dubai comes from its 2025 paper.

Team	Lidar	Camera	IMU / GNSS	State estimation and SLAM	Control	Compute
FS FEUP, Porto	Hesai Pandar	—	Xsens; NTRIP client	KISS-ICP; GTSAM iSAM2 GraphSLAM; EKF-SLAM	Pure pursuit; MPC (acados)	Jetson
Carnegie Mellon Racing	Hesai	ZED2	—	GTSAM iSAM2	—	Jetson
BCN eMotorsport, UPC	Velodyne, Ouster, Hesai, Livox supported	—	VectorNav; SBG	Lidar-inertial odometry[5] (fast_LIMO)	MPC (FORCES Pro)	—
Chalmers FSD	Velodyne VLP-16	—	OxTS; SBG referenced	g2o GraphSLAM	—	Ryzen 7 1700
MUR, Melbourne	Ouster OS1	Basler stereo	Piksi RTK; SBG	FastSLAM; robot_localization	Pure pursuit, then MPCC	Jetson
Ampera, UFSC	Ouster OS1	—	VectorNav	robot_localization; slam_toolbox	—	Jetson

HWU Dubai, Hydrakon. FS-AI. Abdo 2025	RoboSense Helios 16P	ZED2i	CHCNAV INS; ADS-DV CAN	LIO-SAM referenced	Pure pursuit	Jetson AGX Orin 64 GB
FSLART	Lidar, model not named	RealSense; mono camera	Xsens; NTRIP client	g2o GraphSLAM	Pure pursuit; Stanley	Jetson
ISC-FS	Hesai Pandar	—	—	GTSAM GraphSLAM; slam_toolbox	—	—
UoA FSAE, Auckland	—	ZED2	—	—	Stanley; pure pursuit	Jetson

The common patterns

Eight things recur across these stacks. Each is followed by what it means for us.

1. A 3D lidar finds the cones[11]; the camera names their colour[5]

Roughly four in five of the teams in the tables above carry a 3D lidar, and every event winner among them does. The stated reason is range accuracy: a lidar return is good to a few centimetres at any distance, while stereo depth error grows with the square of distance. Vigo, the cleanest camera-only stack[4] here, reports under 0.5 m error only out to 7 m. The lidar's weakness is colour. AMZ could classify colour from intensity only inside 4.5 m, so the camera supplies it.

The Velodyne VLP-16[7] is the most common single model: AMZ's 2017 winner, IIT Bombay and EUFS on the ADS-DV, Zurich UAS, Chalmers and FST Lisboa. IIT Bombay describes it as a "commonly utilized LiDAR unit by FS-AI teams". Three of those pair it with a ZED camera[10].

> **For us:** the Hesai ATX finds the cones and the ZED2 names their colour (Hardware). The owned Puck stays on the rig.

2. Every stack has an IMU[10] and a speed measurement[6]; ground-speed sensors[7] are a top-team luxury

No stack runs without an inertial unit. SBG units appear in at least seven teams, Xsens and VectorNav in two each; most are full GNSS/INS units and many were sponsored. Wheel speeds are used everywhere they exist. The papers report an optical ground speed sensor only on high-power four-wheel-drive cars (AMZ, Elefant), where wheel slip makes wheel speed unreliable.

> **For us:** the MTi-8 kit (IMU). Wheel speeds come from CAN. A 5 kW rear-drive car has little slip, and its undriven front wheels have less, so a ground speed sensor would buy nothing.

3. Nobody steers by GNSS; RTK is a measuring instrument[8]

AMZ won in 2017 with a consumer GPS and says so. In 2020 it used RTK only as ground truth to score its maps. Elefant uses RTK for testing, data collection and laying out tracks, and keeps a second speed source for "the likely case that the GNSS signal fails". Only Vigo and TUW put their own base station in the localisation loop, which the FS-AI rules do not leave room for. Teams that want corrections at an event pull them over a cellular link[2] (Elefant; NTRIP clients in the FEUP and FSLART code).

> **For us:** RTK is ground truth on the rig and an optional aid at the event; the stack must drive without it (GPS).

4. Two filters at two speeds[7]

The strongest stacks split estimation in two. A fast filter turns IMU and speed data into a smooth velocity and short-term pose at 50–200 Hz. A slower mapper corrects that pose against cones at the lidar or camera rate. AMZ runs the first at 200 Hz and the second at 15 Hz. QUT measured the value of the split directly: predicting from a 5 Hz INS alone was the limit on its SLAM, and adding a 50 Hz IMU-and-wheel filter in front improved map accuracy by 65 %.

> **For us:** the odometry EKF at 100 Hz and cone SLAM at the lidar rate (Layers).

5. Map on the first lap, then localise[4]

Cones are landmarks, so teams build a cone map while driving slowly, detect that the lap has closed, freeze the map and then drive faster against it. AMZ mapped at about 3 m/s until 2020. FastSLAM[6] was the early choice (AMZ 2017–19, EUFS, MUR). GraphSLAM[8], usually through GTSAM or g2o, is the later one (AMZ 2020, KIT21d, FEUP, Carnegie Mellon, Chalmers, FSLART). Plain EKF-SLAM[6] is still in use (QUT, FEUP, E-Team Pisa).

> **For us:** trackdrive is ten laps and worth 250 of the 550 dynamic points, so mapping the first lap pays. Autocross is one lap with no stored data, so the exploratory mode must be able to finish a lap by itself.

6. Delaunay mid-line[8] and pure pursuit[8] first, MPC[7] later

For an unknown track, teams triangulate the cones, take the mid-points of the edges that cross the track, and pick the most plausible chain as the centre line (AMZ, KIT, IIT Bombay, BME). The widely reused FaSTTUBe planner in our collection reaches the same mid-line by sorting and matching cones instead. The first controller is almost always pure pursuit: AMZ on its mapping lap, IIT Bombay, QUT, FEUP, MUR, FSLART. Stanley[5] is the other simple choice. MPC arrives later and only once the map and vehicle model are trusted.

> **For us:** pure pursuit with a PI speed loop is enough to score in all four dynamic events at ADS-DV speeds.

7. Jetson-class compute[6] for most, a desktop PC[5] for the fastest

A Jetson appears in about seventeen of the forty-two code bases. Teams running long-horizon MPC at 50 Hz use x86 with a desktop GPU: KIT an i7-9700K, Elefant a Ryzen 3700X with an RTX 2070 Super.

> **For us:** an industrial Intel board with no GPU, the layout of the 2026 winner and of KIT (Computer).

8. Delay is measured and compensated[5], not ignored

AMZ measured about 300 ms from cone detection to command and built delay compensation into both the velocity filter and the mapper. KIT de-skews each 10 Hz lidar sweep because “the distortion caused by the movement while scanning can not be neglected”. AMZ synchronises its two computers with chrony; Elefant uses PTP.

> **For us:** stamp every measurement at acquisition and transform it with the pose from that instant. Every clock on the car follows GNSS (Time).

Who used what

36 sensors and methods, with the teams in the collection that used each one and where it says so. A paper link opens the PDF at the cited page, and any words in quotation marks are on that page. Page numbers are PDF pages,

as the viewer counts them. A *code* link opens the file or folder in this repository; the line number is where to look. Code shows that a driver or module is present, not that it raced. ADS-DV marks FS-AI entries on the car we will use.

Sensors

Velodyne VLP-16 (Puck)

The lidar we own. Two FS-AI entries ran it on the ADS-DV.

IIT Bombay ADS-DV

One VLP-16 with two ZED2i. “Commonly utilized LiDAR unit by FS-AI teams”.

Rampuria 2024, p. 7 ↗

IIT Bombay ADS-DV

Its limit: 2° between 16 rings, so “certain cones (at distance > around 10m) may not be perceived by it”.

Rampuria 2024, p. 1 ↗

EUFS, Edinburgh ADS-DV

VLP-16 and ZED on its ADS-DV model. The file gives the mounting positions it uses.

code: eufs_sim/.../ads-dv/robot.urdf.xacro, line 66 ↗

AMZ *flüela*

Won FSG 2017 with one VLP-16 Puck.

Valls 2018, p. 4 ↗

QUT Motorsport

Velodyne Puck, limited to a 180° field of view.

Bradford 2023, p. 6 ↗

Zurich UAS Racing

VLP-16 on the front spoiler, ZED 2i on the roll cage.

Finn 2025, p. 26 ↗

FST Lisboa

VLP-16 in both its SLAM and its perception thesis.

Lopes 2021, p. 58 ↗

3D lidar as the cone-position sensor

Every event winner in the collection carries one. Models other than the VLP-16:

AMZ 2018–19

Lidar on the front wing, with cameras as an independent second pipeline.

Kabzan 2019, p. 6 ↗

KIT19d

Four lidars and three cameras, facing forward and rearward.

Nekkah 2020, p. 2 ↗

KIT21d

One Hesai Pandar40P at 10 Hz. Cameras are optional.

Alvarez 2022, p. 2 ↗

HWU Dubai ADS-DV
Robosense Helios 16P, a 16-ring unit like ours.
Abdo 2025, p. 1 ↗

BME
Ouster OS1-32 at 20 Hz.
Mező-Kerekes 2026, p. 3 ↗

BCN eMotorsport
Ouster OS1-64.
Carlos 2025, p. 3 ↗

Squadra Corse PoliTo
Velodyne lidar with a ZED stereo camera.
Bertrand 2022, p. 4 ↗

BIT-FSD
3D lidar and a monocular camera.
Chen 2019, p. 2 ↗

FS FEUP
Hesai Pandar driver.
code: hesai-lidar/ ↗

Carnegie Mellon
Hesai AT128 solid-state lidar with two ZED2.
code: PerceptionsDocs/_sources/index.rst.txt, line 24 ↗

MUR, Melbourne
Ouster OS1 driver.
code: OusterDriver/readme.md, line 2 ↗

Ring spacing sets how far a lidar sees a cone

Why a 16-ring lidar is good to roughly 10 m on a 325 mm cone, and what more rings buy.

IIT Bombay ADS-DV
VLP-16: “certain cones (at distance > around 10m) may not be perceived by it”.
Rampuria 2024, p. 1 ↗

AMZ *flüela*
VLP-16: “distant cones can fall in between two Velodyne rays”.
Valls 2018, p. 4 ↗

AMZ 2018–19
Vertical resolution is the parameter that matters most “since it limits the number of returns per cone”.
Kabzan 2019, p. 6 ↗

BME
32 rings: beyond 15 m the lidar “returns fewer than five points per cone cluster”.

Mező-Kerekes 2026, p. 7 ↗

KIT21d
40 rings: cones recognised “at distances of around 35 m”.
Alvarez 2022, p. 1 ↗

Colour comes from the camera, not the lidar

AMZ *flüela*
“Colors cannot be distinguished from the LiDAR point cloud at an acceptable range”.
Valls 2018, p. 4 ↗

AMZ 2018–19
Colour from lidar intensity patterns works only to 4.5 m.
Kabzan 2019, p. 11 ↗

IIT Bombay ADS-DV
YOLOv5 on the image gives the class; lidar points projected into the box give the range.
Rampuria 2024, p. 2 ↗

BME
Camera detections are fused in “to facilitate colour classification”.
Demeter 2025, p. 3 ↗

KIT21d
Runs on a “360° LiDAR and optionally three cameras”.
Alvarez 2022, p. 1 ↗

ZED-family stereo camera

ZED, ZED2 or ZED 2i. In every lidar stack below it supplies colour, not position.

IIT Bombay ADS-DV
Two ZED2i.
Rampuria 2024, p. 1 ↗

HWU Dubai ADS-DV
ZED2i.
Abdo 2025, p. 1 ↗

EUFS, Edinburgh ADS-DV
ZED camera on its ADS-DV model.
code: eufs_sim/.../ads-dv/robot.urdf.xacro, line 70 ↗

QUT Motorsport
ZED2i. Its built-in odometry tracking “proved limited in forward motion”.
Bradford 2023, p. 9 ↗

Zurich UAS Racing
ZED 2i with a 2.1 mm lens, on the roll cage.

Finn 2025, p. 26 ↗

Universidade de Vigo
ZED 2i as the only perception sensor, 1080p at 30 fps.
Fontán-Costas 2025, p. 2 ↗

TUW Racing 2017
ZED, 120 mm baseline.
Zeilinger 2017, p. 3 ↗

Squadra Corse PoliTo
ZED with a Velodyne.
Bertrand 2022, p. 4 ↗

Carnegie Mellon
Two ZED2.
code: PerceptionsDocs/_sources/index.rst.txt, line 25 ↗

UoA FSAE, Auckland
ZED 2i launch in its container setup.
code: autonomous/.docker_templates/docker-compose.CPU.yml, line 40 ↗

Camera-only perception

It works, at shorter range and with larger depth error than lidar.

Universidade de Vigo
ZED 2i alone: median error under 0.5 m, but only out to 7 m.
Fontán-Costas 2025, p. 1 ↗

Elefant Racing
Two Basler cameras near and a 25 cm stereo camera far (0.4 m error at 20 m). A lidar was weighed and put off for a season.
Friedrich 2024, p. 3 ↗

MIT / Delft
Stereo and mono cameras with YOLOv3. A lidar is carried only as an isolated safety backup.
Strobel 2020, p. 1 ↗

UoA FSAE, Auckland
ZED-based stack.
code: autonomous/README.md, line 5 ↗

Intel RealSense

One code base in the collection has a driver node. No paper reports racing one.

FSLART
RealSense node built on librealsense2.
code: WatchedRealSense/src/realsense_node.cpp, line 4 ↗

2D (planar) lidar

One stack in the collection. It sat in the low zone ahead of the front tyres, which teams on a shared ADS-DV may not use.

TUW Racing 2017
Hokuyo planar scanner “inside the front wing, at the lowest point, as it is a planar scanner and the cones are relatively short”, tilted down to offset pitch under acceleration and braking.
Zeilinger 2017, p. 3 ↗

IMU or INS on the car

No stack runs without one. Most are GNSS/INS units, and several were sponsored.

IIT Bombay ADS-DV
SBG Ellipse-N GNSS/INS, “acquired as a sponsored unit”.
Rampuria 2024, p. 7 ↗

HWU Dubai ADS-DV
CHCNAV INS.
Abdo 2025, p. 1 ↗

EUFS, Edinburgh ADS-DV
IMU and GPS on its ADS-DV model.
code: eufs_sim/.../ads-dv/robot.urdf.xacro, line 58 ↗

AMZ *flüela*
INS “combining an IMU and a GPS”.
Valls 2018, p. 1 ↗

QUT Motorsport
SBG Ellipse-D INS.
Bradford 2023, p. 6 ↗

Zurich UAS Racing
SBG Ellipse-D.
Finn 2025, p. 30 ↗

Squadra Corse PoliTo
SBG Ellipse-N.
Bertrand 2022, p. 4 ↗

KIT19d
IMU and wheel speed sensors feed localisation.
Nekkah 2020, p. 2 ↗

Elefant Racing
IMU inside the Novatel GNSS receiver.
Friedrich 2024, p. 3 ↗

FS FEUP
Xsens MTi-680 driver.
code: xsens-imu/README.md ↗

Wheel speeds in the estimator

Used wherever the car has them. On the ADS-DV they arrive on CAN.

EUFS, Edinburgh ADS-DV
Its bridge publishes the ADS-DV wheel speeds from CAN.
code: EUFS__ros_can/README.md, line 39 ↗

AMZ *flüela*
Reads each wheel encoder.
Valls 2018, p. 2 ↗

KIT19d
Rear wheel speeds and yaw rate drive the EKF prediction directly.
Nekkah 2020, p. 6 ↗

QUT Motorsport
Hall-effect wheel speed and steering angle sensors.
Bradford 2023, p. 6 ↗

BIT-FSD
GPS/INS and wheel speed sensors.
Chen 2019, p. 2 ↗

Elefant Racing
The exception: deliberately none.
Friedrich 2024, p. 3 ↗

Optical ground speed sensor

Fitted where wheel speed cannot be trusted: four driven wheels and high slip. The ADS-DV has 5 kW on the rear axle and free-rolling front wheels.

AMZ *flüela*
Optical ground speed sensor on the 4WD car that won FSG 2017.
Valls 2018, p. 2 ↗

AMZ 2018
The reason: race cars have “high wheel slip, up to 20% for optimal longitudinal acceleration”.
Gosala 2018, p. 3 ↗

AMZ 2018–19
The car: “an electric 4WD racecar”, 190 kg, 0–100 km/h in 1.9 s.
Kabzan 2019, p. 6 ↗

Elefant Racing

Kistler Correvit SFII on a car with four wheel-hub motors, to get redundancy “without estimating tire slip”.

Friedrich 2024, p. 3 ↗

AMZ 2020

Calls them “expensive external velocity sensors” and trains a network to do without.

Srinivasan 2020, p. 1 ↗

Zurich UAS Racing

Models a Kistler Correvit in its simulator.

code: driverless-simulator/docs/ground-speed-sensor.md, line 3 ↗

FSLART

Firmware project named GroundSpeedSensor, undocumented.

code: GroundSpeedSensor/ ↗

GNSS and RTK

Used for testing and ground truth. At the event there is no room for a base station.

FS-AI rule T4.5.2

“(D)GPS may be used but there will be no space to securely build up base stations”.

FS-AI rules 2026, p. 15 ↗

AMZ *flüela*

Won with consumer GPS: “no differential GPS or RTK”.

Valls 2018, p. 2 ↗

AMZ 2020

RTK only as ground truth for scoring maps.

Andresen 2020, p. 5 ↗

Elefant Racing

Dual-antenna RTK when reception is good, with a second speed source for “the likely case that the GNSS signal fails”.

Friedrich 2024, p. 2 ↗

Universidade de Vigo

ZED-F9P at 10 Hz with a fixed base up to 11 km away, inside the localisation loop.

Fontán-Costas 2025, p. 2 ↗

TUW Racing 2017

Piksi Multi with “two beacons placed outside the racetrack”.

Zeilinger 2017, p. 3 ↗

Zurich UAS Racing

A GNSS-RTK INS was its most recent addition.

Finn 2025, p. 21 ↗

MUR, Melbourne

Piksi RTK driver.

code: piksi/ ↗

RTK corrections over a cellular link

Elefant Racing
Corrections from the internet through a “dual-SIM cellular router”, also used for telemetry.
Friedrich 2024, p. 2 ↗

FS FEUP
NTRIP client inside its Xsens driver.
code: xsens-imu/.../src/ntrip_util.cpp ↗

Two GNSS antennas for heading

Gives heading at standstill. The teams below bought it inside an INS.

AMZ 2018–19
Dual-antenna INS, heading accurate to 0.08°.
Kabzan 2019, p. 7 ↗

Elefant Racing
Novatel PwrPak7D-E2.
Friedrich 2024, p. 2 ↗

Zurich UAS Racing
SBG Ellipse-D with front and rear antennas.
Finn 2025, p. 30 ↗

QUT Motorsport
SBG Ellipse-D, the dual-antenna model.
Bradford 2023, p. 6 ↗

More than one lidar

KA-RaceIng went from four lidars to one and won with the one. A BME paper on dual-lidar placement is on the manual-download list in index.html.

KIT19d
Four lidars, forward and rearward, to detect “at ranges up to 42 m around the vehicle”.
Nekkah 2020, p. 2 ↗

KIT21d
Two years later: “one Hesai Pandar40P”.
Alvarez 2022, p. 2 ↗

Radar

No team stack in the collection has a radar driver in its launch or config files.

AMZ 2018–19

“A LiDAR was chosen over radars because of its ability to detect small non-metallic objects (such as plastic cones)”.

Kabzan 2019, p. 6 ↗

Estimation and mapping

A fast odometry filter ahead of the mapper

AMZ 2018
Slip-aware EKF for velocity. SLAM propagates on its 200 Hz output and corrects with cones at 15 Hz.
Gosala 2018, p. 4 ↗

QUT Motorsport
A separate robot_localization filter before EKF landmark SLAM “improved map accuracy by 65% on average and pose tracking by 81%”.
Bradford 2023, p. 6 ↗

QUT Motorsport
Its filter configuration: 50 Hz.
code: QUTMS_Driverless/…/config/localisation_params.yaml, line 12 ↗

KIT19d
EKF on pose, predicted from rear wheel speeds and yaw rate.
Nekkah 2020, p. 6 ↗

BIT-FSD
EKF localisation for “high accuracy and high frequency state estimation”.
Chen 2019, p. 1 ↗

FS FEUP
Separate EKF state estimator configuration.
code: autonomous-systems/…/ekf_state_est/vehicle.yaml, line 1 ↗

Universidade de Vigo
The counter-example: one EKF stepped at 0.1 s “because the RTK has a maximum refresh rate of 10Hz”.
Fontán-Costas 2025, p. 4 ↗

Delay compensation and time sync

AMZ *flüela*
Delay compensation built into the velocity filter.
Valls 2018, p. 3 ↗

AMZ 2018–19
Cone detection to command “sums up to about 300 ms”.
Kabzan 2019, p. 34 ↗

KIT21d
De-skews each sweep: “the distortion caused by the movement while scanning can not be neglected”.

Alvarez 2022, p. 3 ↗

AMZ *flüela*
Chrony keeps its two computers on one clock.
Valls 2018, p. 2 ↗

Elefant Racing
PTP between processing units; one unit triggers all cameras.
Friedrich 2024, p. 3 ↗

Map on lap one, then localise

AMZ *flüela*
“The switch from SLAM to localization is performed after a loop closure”.
Valls 2018, p. 4 ↗

AMZ 2018–19
Lap one on a pure pursuit mid-line; the finished map then goes to the MPC.
Kabzan 2019, p. 5 ↗

AMZ 2020
Raised mapping speed on an unknown track “from 3 m/s to 12 m/s”.
Andresen 2020, p. 1 ↗

KIT19d
Builds its global map during the autocross run.
Nekkah 2020, p. 6 ↗

EKF-SLAM
One state vector, easiest to debug. The proposed first mapper.

QUT Motorsport
EKF landmark SLAM “has been previously employed by the QUT Motorsport team”.
Bradford 2023, p. 2 ↗

TUW Racing 2017
EKF-SLAM on cone measurements.
Zeilinger 2017, p. 4 ↗

KIT19d
Simpler still: an EKF on pose only, where “the landmark positions are not included in the state vector”.
Nekkah 2020, p. 6 ↗

FS FEUP
Solver option `ekf_slam` beside `graph_slam`.
code: autonomous-systems/…/slam/pacsim.yaml, line 25 ↗

E-Team Pisa
EKF SLAM package for cone mapping.

code: etdv-ekf-slam/package.xml, line 5 ↗

UoA FSAE, Auckland
Kalman-filter landmark mapping.
code: autonomous/README.md, line 5 ↗

FastSLAM (particle filter)

AMZ *flüela*
Chosen because cones look alike: “we choose to use FastSLAM”, which handles data association per particle.
Valls 2018, p. 5 ↗

AMZ 2018–19
Moved to FastSLAM 2.0.
Kabzan 2019, p. 4 ↗

EUFS, Edinburgh ADS-DV
“FastSLAM 2.0 and GraphSLAM have been implemented and tested on real-world track data”.
Newlands 2024, p. 2 ↗

Elefant Racing
Picked because it is “easier to understand than other common SLAM algorithms”.
Friedrich 2024, p. 4 ↗

FST Lisboa
Implemented “two particle filter approaches and a graph-based one”.
Lopes 2021, p. 9 ↗

MUR, Melbourne
FastSLAM 2.0 tutorial for its own implementation.
code: 2024_2025_av_sim_stack/.../tutorials/tut_slam.rst, line 1 ↗

GraphSLAM

The later choice of the fastest teams, usually through GTSAM or g2o.

AMZ 2020
A “pose-landmark GraphSLAM algorithm” running on the car.
Andresen 2020, p. 4 ↗

KIT21d
Built on g2o.
Alvarez 2022, p. 4 ↗

KA-RaceIng thesis
Head to head: “GraphSLAM outperforms EKF SLAM in terms of accuracy”, at a higher computing cost.
Le Large 2020, p. 7 ↗

EUFS, Edinburgh ADS-DV

Compared with FastSLAM 2.0 on real track data.
Newlands 2024, p. 2 ↗

FS FEUP
GTSAM iSAM2 as the optimiser.
code: autonomous-systems/.../slam/pacsim.yaml, line 28 ↗

Carnegie Mellon
iSAM2 SLAM repository.
code: iSAM2_SLAM/ ↗

ISC-FS
GTSAM cone graph, replacing dense lidar-inertial SLAM.
code: IFS09-DV-PIPELINE/cone_slam/package.xml, line 8 ↗

FSLART
Graph SLAM repository.
code: graph_slam/ ↗

Lidar-inertial odometry

Pose from scan matching instead of cone landmarks. Not part of the proposed stack.

BCN eMotorsport
Its own lidar-inertial odometry SLAM, under 20 ms per scan on an Ouster OS1-64.
Carlos 2025, p. 3 ↗

BME
Based on FAST-LIO2, adopted after its earlier approach reached its limits.
Demeter 2025, p. 3 ↗

BIT-FSD
Lidar odometry and GPS-INS “coupled by complementary” filtering.
Tian 2020, p. 5 ↗

FS FEUP
KISS-ICP fork.
code: kiss-icp/ ↗

BCN eMotorsport
fast_LIMO, with Velodyne, Ouster, Hesai and Livox supported.
code: fast_LIMO/config/cat.yaml, line 16 ↗

Perception methods

Lidar cone detection: remove the ground, then cluster

AMZ 2018–19
Adaptive ground removal, then “Euclidean clustering and classification based on geometric priors”.

Kabzan 2019, p. 4 ↗

KIT21d
Crops the cloud (60 % fewer points), removes the ground plane (a further 30 %), then de-skews.
Alvarez 2022, p. 3 ↗

FST Lisboa
Finds that the published pipelines share these two first steps, and builds its own the same way.
Morgado, p. 31 ↗

IIT Bombay ADS-DV
Clusters first, so that only cone points are projected into the image.
Rampuria 2024, p. 2 ↗

BIT-FSD
Ground plane by RANSAC.
Tian 2020, p. 4 ↗

BME
A Random Forest classifies the clusters, on the CPU, at 20 Hz.
Mező-Kerekes 2026, p. 3 ↗

FSLART
RANSAC ground removal, DBSCAN clustering, then a small CNN classifies the image crop of each cluster.
code: local_mapping_lidar/README.md, line 5 ↗

Camera cone detection with a YOLO-type network

AMZ 2018–19
“a modified version of YOLOv2”, followed by keypoint regression.
Kabzan 2019, p. 4 ↗

MIT / Delft
YOLOv3.
Strobel 2020, p. 1 ↗

IIT Bombay ADS-DV
YOLOv5.
Rampuria 2024, p. 1 ↗

Universidade de Vigo
YOLOv11, 0.93 mAP@0.5 on FSOCO.
Fontán-Costas 2025, p. 1 ↗

Squadra Corse PoliTo
YOLOv4 compared with SSD.
Berettoni 2021, p. 3 ↗

KIT19d
A classifier instead: “an adapted MobileNet V2”.

Nekkah 2020, p. 6 ↗

FSOCO
The shared cone image dataset that teams contribute to and train on.
Vödisch 2020, p. 1 ↗

MUR, Melbourne
YOLO under tkDNN.
code: tkdnn_yolo_ros/ ↗

Lidar position plus camera colour

IIT Bombay ADS-DV
Lidar clusters projected into YOLO boxes: “the combined approach is novel for the purpose of Formula Student AI”.
Rampuria 2024, p. 1 ↗

HWU Dubai ADS-DV
Lidar–camera fusion node.
code: Formula-Student-AI/.../hydrakon_simulation/lidar_camera_fusion.py, line 55 ↗

KIT21d
Lidar detections become image boxes that are “passed on to the image classification system”.
Alvarez 2022, p. 3 ↗

AMZ 2020
“early fusion of camera and LiDAR data”.
Andresen 2020, p. 1 ↗

BME
Camera detections add colour to lidar cones.
Demeter 2025, p. 3 ↗

FSLART
Crops the camera image around each lidar cluster and classifies the crop.
code: local_mapping_lidar/README.md, line 5 ↗

Calibration method
Lidar–camera extrinsics from 3D–3D point correspondences.
Dhall 2017, p. 1 ↗

Elefant Racing
The dissent: judged projecting scanning-lidar points into images too complex for a small team.
Friedrich 2024, p. 3 ↗

Planning and control

Mid-line from Delaunay triangulation

AMZ 2018–19
Triangulates the cones, samples paths through edge mid-points, re-plans every 50 ms.
Kabzan 2019, p. 22 ↗

KIT21d
Refined “the algorithm based on the Delaunay Triangulation developed last year”.
Alvarez 2022, p. 4 ↗

IIT Bombay ADS-DV
Replaced distance-threshold cone pairing with Delaunay, “a more robust version”.
Rampuria 2024, p. 5 ↗

BME
“Delaunay triangulation is used on the cone map”, for every event but skidpad.
Demeter 2025, p. 4 ↗

AMZ 2020
Delaunay with Bayesian inference over candidate paths.
Andresen 2020, p. 3 ↗

BCN eMotorsport
Track limits and centre line from Delaunay.
code: urinay/package.xml, line 5 ↗

FSLART
Delaunay planner library.
code: delaunay_path_planner_core/ ↗

FaSTTUBe
The alternative: the same mid-line by sorting and matching cones, with no triangulation, and it works when colours are unknown. Auckland and FSLART reuse it.
code: ft-fsd-path-planning/README.md, line 52 ↗

Pure pursuit

The usual first controller.

IIT Bombay ADS-DV
Cross-track error fell “from 0.33 to 0.27 from Stanley control to Pure Pursuit control” in simulation.
Rampuria 2024, p. 6 ↗

HWU Dubai ADS-DV
Pure pursuit parameters in its fusion node.
code: Formula-Student-AI/.../hydrakon_simulation/lidar_camera_fusion.py, line 55 ↗

AMZ 2018–19
Tracks the mid-line on lap one.

Kabzan 2019, p. 5 ↗

QUT Motorsport
Its own pure pursuit and Nav2 regulated pure pursuit, compared.
Bradford 2023, p. 8 ↗

Universidade de Vigo
Regulated pure pursuit, look-ahead set by speed error.
Fontán-Costas 2025, p. 1 ↗

FS FEUP
Pure pursuit beside an MPC.
code: autonomous-systems/…/control/pacsim.yaml, line 3 ↗

MUR, Melbourne
PID plus pure pursuit follower.
code: path_follower_ros/src/main.py, line 10 ↗

FSLART
Pure pursuit core library.
code: pure_pursuit_core/README.md, line 1 ↗

Stanley controller

BME
Began with “the Stanley controller”.
Demeter 2025, p. 5 ↗

IIT Bombay ADS-DV
Replaced by pure pursuit after a simulator comparison.
Rampuria 2024, p. 6 ↗

UoA FSAE, Auckland
Stanley lateral control.
code: autonomous/README.md, line 5 ↗

FSLART
Stanley steering calculation.
code: driverless_pipeline/…/math/stanley_test.cpp, line 59 ↗

Ampera, UFSC
Stanley controller repository.
code: StanleyControllerMethod-SCM-/ ↗

Model predictive control

Arrives after the map and the vehicle model are trusted.

AMZ 2018–19
After lap one: 50 ms step, “a prediction horizon of 40 time steps”, 2 s ahead.

Kabzan 2019, p. 30 ↗

KIT19d
20 ms step, 65 steps.
Nekkah 2020, p. 9 ↗

KIT21d
MPC for steering, PI for speed.
Alvarez 2022, p. 4 ↗

BIT-FSD
MPC path tracking.
Tian 2020, p. 7 ↗

FS FEUP
MPC through acados.
code: autonomous-systems/.../lateral_controller/mpczinho.cpp, line 8 ↗

BCN eMotorsport
MPC through FORCES Pro.
code: tailored_mpc/ ↗

MUR, Melbourne
Contouring MPC.
code: mpcc/ ↗

Race line once the map is closed

KIT21d
Minimum-curvature line: “a reduction in lap time of more than 10% can be achieved”.
Alvarez 2022, p. 4 ↗

Universidade de Vigo
Racing line by cubic splines, with a curvature-limited speed profile.
Fontán-Costas 2025, p. 1 ↗

TUM (reference)
Open minimum-curvature and minimum-time optimiser.
code: TUMFTM__global_racetrajectory_optimization/ ↗

Compute and car interface

Jetson compute

Both FS-AI papers that list their hardware name an AGX Orin.

IIT Bombay ADS-DV
Jetson AGX Orin, with a VLP-16 and two ZED2i. A DDT entry.
Rampuria 2024, p. 1 ↗

HWU Dubai ADS-DV
Jetson AGX Orin 64 GB.
Abdo 2025, p. 1 ↗

Universidade de Vigo
Orin NX, for a camera-only stack.
Fontán-Costas 2025, p. 1 ↗

Zurich UAS Racing
Xavier AGX and Orin NX.
Finn 2025, p. 11 ↗

MIT / Delft
Xavier for detection and depth.
Strobel 2020, p. 3 ↗

Squadra Corse PoliTo
Xavier.
Santacroce 2021, p. 4 ↗

x86 PC, often with a desktop GPU

The choice of teams running long-horizon MPC at 50 Hz.

KIT19d
Intel i7-9700K, 32 GB.
Nekkah 2020, p. 3 ↗

KIT21d
Multi-core x86, plus a Coral Edge TPU when cameras are used.
Alvarez 2022, p. 2 ↗

Elefant Racing
Ryzen 3700X with an RTX 2070 Super.
Friedrich 2024, p. 2 ↗

BIT-FSD
Industrial computer with a GTX 1050 Ti.
Chen 2019, p. 2 ↗

AMZ 2018–19
A rugged industrial master computer and an embedded slave.
Kabzan 2019, p. 7 ↗

CAN bridge to the ADS-DV

IMechE ADS-DV
Official C library over SocketCAN.
code: FS-AI_API/ ↗

EUFS, Edinburgh ADS-DV

ROS 2 wrapper around that library, 100 Hz by default.

code: EUFS__ros_can/README.md, line 52 ↗

Newcastle Racing AI ADS-DV

Its own CAN API package.

code: NewcastleRacingAI__nrai_ADS-DV_CAN-API/ ↗

IIT Bombay ADS-DV

“Interface between the hardware and software is done using ROS2 & CAN communication”.

Rampuria 2024, p. 1 ↗

Simulating the ADS-DV

EUFS, Edinburgh ADS-DV

eufs_sim: the ADS-DV with a VLP-16, a ZED, an IMU and a GPS.

code: eufs_sim/.../ads-dv/robot.urdf.xacro, line 66 ↗

IIT Bombay ADS-DV

Used the EUFS simulator and FSDS “for simulations in all subsystems”.

Rampuria 2024, p. 7 ↗

HWU Dubai ADS-DV

Stack “validated extensively in CARLA” on an ADS-DV model before moving to hardware.

Abdo 2025, p. 1 ↗

Newcastle Racing AI ADS-DV

Its fork of eufs_sim.

code: eufs_sim/ ↗

Heartbeats and fault handling

AMZ *flüela*

The real-time ECU “monitors the heartbeat of the autonomous master computer”; a safety layer on the master watches every software module.

Valls 2018, p. 6 ↗

AMZ *flüela*

A chi-square test on every sensor rejects outliers and doubles as a health estimate.

Valls 2018, p. 3 ↗

FSLART

Safety monitor repository.

code: safety_monitor/ ↗

Sources are in this repository: papers by author and year under `02_fsd_papers/`; code under `01_fsd_open_source_code/team_stacks/`; ADS-DV rules, CAN database and compute guide under `03_fsuk_ddt_ads-dv/`.

Sources

- CarMaker sensors and ROS 2 interface:
  - `FSAI_UsersGuide.md` (sections 2 and 5.1) · GitHub
  - vehicle file `FS-AI_IMechE_ADS-DV_2026`
  - lidar beam table `LidarRSI_FS_autonomous`
  - repository notes `AGENTS.md`
- AS states, mission status and emergency stop: `ADS-DV_2026_CAN1_no_GPS.dbc` (signals `AS_STATE`, `MISSION_STATUS`, `ESTOP_REQUEST`)
- FSDS:
  - GitHub repository (ROS 2 bridge; last push 13 September 2025)
  - v2.2.0 documentation (sensor list)
- HWU Dubai's CARLA stack: Abdo 2025, arXiv 2511.11310
- ROS 2 over Wi-Fi: Stereolabs, Clearpath
- Team evidence:
  - Rampuria 2024 (IIT Bombay)
  - Trinity Rigby
  - UWE-AI on DesignSpark
  - OzU CarMaker deck
  - How to FS-AI (EUFS)
- FS-AI official results: 2022, 2023, 2024, 2025, 2026
- IPG free CarMaker licence for FS-AI teams: IPG, 2025
- FS-AI 2026 rules and the APC technical rules, from the IMechE forms and documents page. The APC needs a wheelbase of at least 1400 mm, so a scaled car cannot be entered as an APC.
- FSG Driverless Cup team pages (Driverless System field), fetched 19 October 2026: 2026 list, e.g. TU Hamburg, NTNU, ETH. The 2027 list has no teams yet, and the Driverless Cup needs a team's own driverless car.
- Zurich UAS Racing's use of the Ellipse-D: SBG case study and Finn 2025
- Xsens leaflets: MTi-8, MTi-3, MTi-630
- Official cone supplier, named in the FS-AI cone specification: WEMAS via baustellenabsicherung24.de
- CARLA physics fields: CARLA 0.9.15 control.yml

The figures are generated by a script with layout checks: no text overflows a box, no arrow passes through a box it does not connect, and no labels overlap.